



6

40

6000 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة-النظم والشبكات

عملية إعادة الإرسال

في TCP

د. علي حماد



RB Informatics; 06/05/2026

بروتوكولات الإتصال

Round-Trip Time Estimation and Timeout

نعلم أن من أهم ميزات بروتوكول ال TCP أنه يعيد إرسال الرسائل في حال ضياعها ولكن الأمر ليس بهذه السهولة فيواجهة ال TCP تحدياً كبيراً وهو متى أقرر أن الرسالة ضاعت وعلي إعادة إرسالها ؟ إن قرر بسرعة إعادة الإرسال يمكن أن يرسل رسائل لم تضع أصلاً . و إن إنتظر طويلاً ليتأكد من ضياع الرسالة تأخر بالرد و ضعف الأداء . ما الحل ☹️ ؟ هنا يأتي دور ال Timeout Mechanism في بروتوكول TCP ليحل هذه المشكلة .

ما هو Timeout Mechanism ؟

هو آلية المهمة الزمنية أو إعادة الإرسال (timeout/retransmit mechanism) يستخدمه بروتوكول TCP للتعافي من الأجزاء المفقودة (lost segments) . أي (سنحدد وقت عند انتهاءه أعيد الإرسال هذا الوقت هو Timeout).

شروطه :

يجب أن تكون المهمة الزمنية أكبر من وقت الذهاب والإياب (RTT) الخاص بالاتصال أي الوقت المستغرق من لحظة إرسال الجزء وحتى استقبال تأكيد وصوله (acknowledged) .

و لكن ما هو وقت الذهاب و الإياب (RTT) :

يُرمز له ب SampleRTT: هي مقدار الوقت بين لحظة إرسال الجزء Segment (أي تمريره إلى بروتوكول IP) ولحظة استقبال تأكيد وصول هذا الجزء.

كيف نقيس الوقت (SampleRTT) ؟

عندما نرسل حزمة (Segment) ، نبدأ بحساب الوقت من لحظة خروجها حتى لحظة وصول الرد (Acknowledgment) هذا الوقت الفعلي الذي نقيسه في كل مرة نسميه SampleRTT.

ملاحظة هامة : إذا اضطررنا لإعادة إرسال الحزمة، لا نحسب لها SampleRTT ، لأننا لن نعرف هل الرد الذي وصل كان للحزمة الأولى أم الثانية.

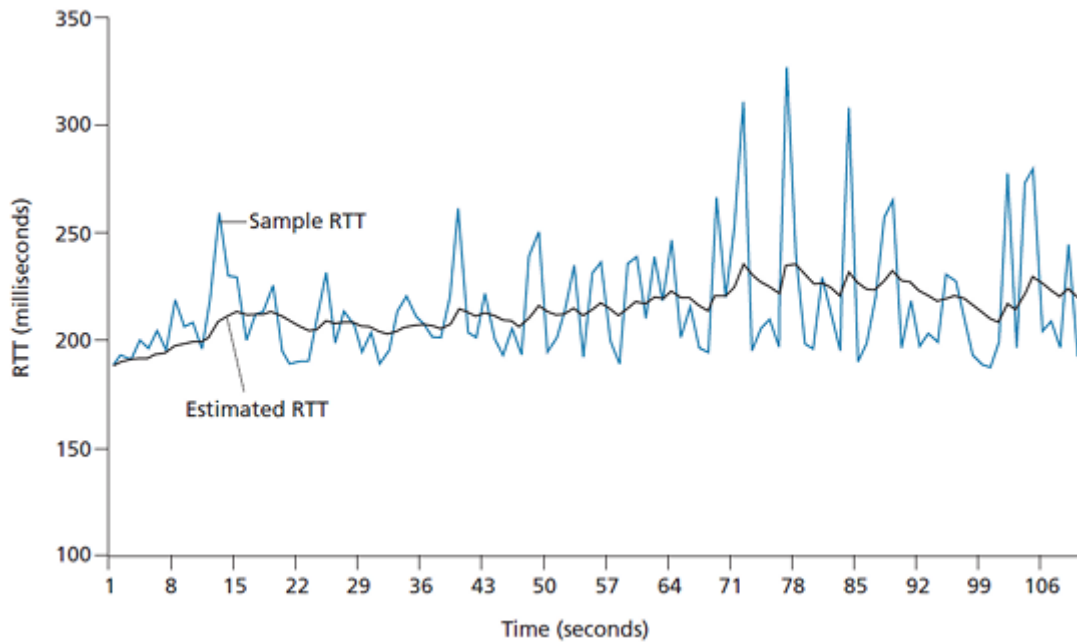
تتغير قيم SampleRTT من Segment إلى أخرى بسبب:

1. الازدحام في أجهزة التوجيه (Routers)
2. اختلاف الحمل على الأجهزة الطرفية
3. تغيير حالة الشبكة لحظة بلحظة

لذلك لا يمكن الاعتماد على قيمة واحدة فقط، بل يجب حساب متوسط يمثل RTT الحقيقي بشكل أفضل. TCP يحتفظ بقيمة متوسطة تسمى **EstimatedRTT**، ويتم تحديثها عند وصول كل SampleRTT جديدة باستخدام المعادلة:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

- القيمة الجديدة لـ **EstimatedRTT** هي عبارة عن مزيج موزون (weighted combination) بين القيمة السابقة لـ **EstimatedRTT** والقيمة الجديدة لـ **SampleRTT**.
- حيث أن القيمة الموصى بها لـ α هي 0.125 (أي 1/8). هذه المعادلة تجعل التحديث تدريجياً بحيث لا يتغير **EstimatedRTT** بسرعة كبيرة ولا يبقى ثابتاً بشكل مبالغ فيه.



يمثل هذا الشكل البياني مثالاً لقياسات **SampleRTT** و **EstimatedRTT** في اتصال TCP بين خادم في جامعة ماساتشوستس - gaia.cs.umass.edu - في الولايات المتحدة، وخادم آخر في جنوب فرنسا - fantasia.eurecom.fr - وقد تم استخدام القيمة $\alpha = 1/8$ في حساب **EstimatedRTT** كما عرفها RFC 5681. في هذا المثال نلاحظ ما يلي:

الخط الأزرق (SampleRTT) :

يعرض القيم الفعلية لزمن الرحلة RTT لكل Segment هذه القيم تتذبذب بشكل واضح، فترتفع وتنخفض بسبب تغيير حالة الشبكة، الازدحام، أو اختلاف الحمل على الأجهزة.

الخط الأسود (EstimatedRTT) :

يمثل المتوسط المحسوب باستخدام المعادلة:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

هذا الخط أكثر سلاسة لأنه لا يتبع التذبذب اللحظي، بل يدمج القيم الجديدة تدريجياً مع القيم السابقة.

نلاحظ أن EstimatedRTT لا يتغير بسرعة مثل SampleRTT ، وهذا هو الهدف: الحصول على قيمة مستقرة تساعد TCP على اتخاذ قرار صحيح حول وقت إعادة الإرسال.

↪ التعامل مع عدم الاستقرار "DevRTT" :

إضافةً إلى حساب متوسط RTT ، يحتاج TCP إلى قياس مدى تذبذب قيم SampleRTT ، لأن هذا التذبذب يؤثر مباشرة على حساب قيمة Timeout فإذا كان الوقت مستقرًا (مثلاً دائماً حول 50 ms) فالشبكة ممتازة. أما إذا كان يقفز بين 10 ms و 200 ms، فهناك عدم استقرار كبير.

هذا التذبذب نقيسه بما يسمى DevRTT (تطور الانحراف) لذلك يعرف RFC 6298 قيمة DevRTT :

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

القيمة الموصى بها للمعامل β هي 0.25

وهنا نميز حالتين :

- إذا كانت قيم الـ SampleRTT تحتوي على تقلبات وتذبذبات طفيفة ----- < فإن قيمة DevRTT ستكون صغيرة.
- إذا كان هناك الكثير من التقلبات ----- < فإن قيمة DevRTT ستكون كبيرة.

فائدة أربيسيزية:

RFC 5681 هو وثيقة معيارية من IETF تصف آليات التحكم في الازدحام في TCP ، وهي مجموعة قواعد تمنع الشبكات من الانهيار عندما يرسل العديد من الأجهزة بيانات في نفس الوقت. الوثيقة تشرح كيف يجب على TCP أن يزيد أو يقلل معدل الإرسال اعتماداً على حالة الشبكة.

RFC 6298 هو المعيار الذي يحدد الطريقة الرسمية لحساب وقت إعادة الإرسال (RTO) في بروتوكول TCP ، وذلك باستخدام قياسات RTT ومتغيراته لضمان إعادة الإرسال بشكل دقيق دون مبالغة أو تأخير.

بعد أن تعلمنا كيف نحسب المتوسط (EstimatedRTT) وكيف نعرف مدى التذبذب (DevRTT) ، نأتي للسؤال الأهم، كم

ننتظر بالضبط قبل أن نقول "الحزمة ضاعت" ونعيد إرسالها؟

بروتوكول TCP يعتمد على قيمة TimeoutInterval لتحديد الوقت الذي يجب أن ينتظره قبل أن يعيد إرسال Segment لم يصل ACK الخاص بها.

وهنا تظهر مشكلة حساسة جداً:

- إذا كان Timeout أصغر من EstimatedRTT ← سيعيد TCP الإرسال بلا داع.
 - إذا كان Timeout أكبر بكثير من EstimatedRTT ← لن يقوم TCP بإعادة إرساله بسرعة، مما يؤدي إلى تأخيرات كبيرة في نقل البيانات.
- إذن يجب أن تكون قيمة Timeout متوازنة، لذلك TCP لا يعتمد فقط على EstimatedRTT ، بل يضيف إليه هامش أمان (margin) . يجب أن يكون الهامش كبيراً عندما يكون هناك الكثير من التقلبات والتذبذبات في قيم SampleRTT ويجب أن يكون صغيراً عندما تكون التقلبات طفيفة. وبالتالي، ينبغي أن تدخل قيمة DevRTT في الحساب هنا:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

لماذا نضيف DevRTT ؟

- إذا كانت الشبكة مستقرة ← DevRTT صغير ← الهامش صغير ← Timeout قريب من EstimatedRTT .
 - إذا كانت الشبكة متذبذبة ← DevRTT كبير ← الهامش أكبر ← Timeout يصبح أكثر مرونة.
- بهذا الشكل يتكيف TCP مع حالة الشبكة لحظة بلحظة.

ملاحظات مهمة:

- القيمة الابتدائية لـ TimeoutInterval يجب أن تكون 1 ثانية.
 - عند حدوث Timeout (أي لم يصل ACK في الوقت المحدد) يقوم TCP بمضاعفة TimeoutInterval
- الهدف:
- تجنب إعادة الإرسال بسرعة
 - إعطاء الشبكة فرصة للتعافي من الازدحام
- بمجرد وصول ACK جديد وتحديث EstimatedRTT ، يعيد TCP حساب TimeoutInterval مباشرة باستخدام المعادلة الأساسية.

Reliable Data Transfer in TCP

بروتوكول IP لا يضمن وصول البيانات دائماً، فهو يعمل بأسلوب Best-Effort فقط فقد يحدث ضياع للـ datagrams بسبب امتلاء الـ buffers ووصول البيانات بترتيب خاطئ أو حتى تلف بعض البتات أثناء النقل.

وبما أن TCP يعتمد على IP لنقل الـ segments ، فهو معرض لنفس المشاكل، لكن TCP لا يقبل بهذه الفوضى، بل يبني فوق IP خدمة نقل بيانات موثوقة.

مهمة الـ TCP هي تحويل هذه الفوضى إلى "خدمة موثوقة" عندما يقرأ التطبيق (Application) البيانات من الـ TCP ، فإنه يضمن أربعة أشياء:



1. غير تالف Uncorrupted: البيانات سليمة تماماً ولم تتغير.
2. بلا فجوات Without Gaps: لا توجد حزم مفقودة.
3. دون تكرار Without Duplication: لا توجد حزم مكررة.
4. بالترتيب In Sequence: البيانات تصل بالترتيب الصحيح تماماً كما أرسلت.

الأحداث الرئيسية في عملية الإرسال داخل TCP Sender

هناك ثلاثة أحداث أساسية يتعامل معها TCP أثناء الإرسال:

1 استقبال بيانات من التطبيق (Application Data Arrival)

يستقبل TCP البيانات من التطبيق ويضعها داخل Segment ، ثم يمرّر الـ Segment إلى IP لإرساله عبر الشبكة. إذا لم يكن هناك مؤقت يعمل، يبدأ TCP تشغيل المؤقت (Timer) وهذا المؤقت مرتبط دائماً بأقدم Segment لم يصل ACK لها.

فاصل انتهاء الصلاحية لهذا المؤقت هو TimeoutInterval ، والذي يتم حسابه من قيمتي EstimatedRTT و DevRTT.

2 انتهاء المؤقت (Timeout Event)

عند انتهاء المؤقت يعيد TCP إرسال الـ Segment الذي لم يصل ACK له ويعيد تشغيل المؤقت مرة أخرى. وتكون قيمة المؤقت هي TimeoutInterval المحسوبة من EstimatedRTT و DevRTT .

3 استلام ACK من المستقبل (ACK Arrival)

عند وصول ACK يقارن TCP قيمة الـ ACK (نسميها y) مع المتغير SendBase (وهو رقم تسلسل أقدم بايت لم يتم تأكيده بعد). وبالتالي فإن $SendBase - 1$ هو رقم التتابع لآخر بايت عُرف أنه تم استلامه بشكل صحيح وبالترتيب عند المستقبل.

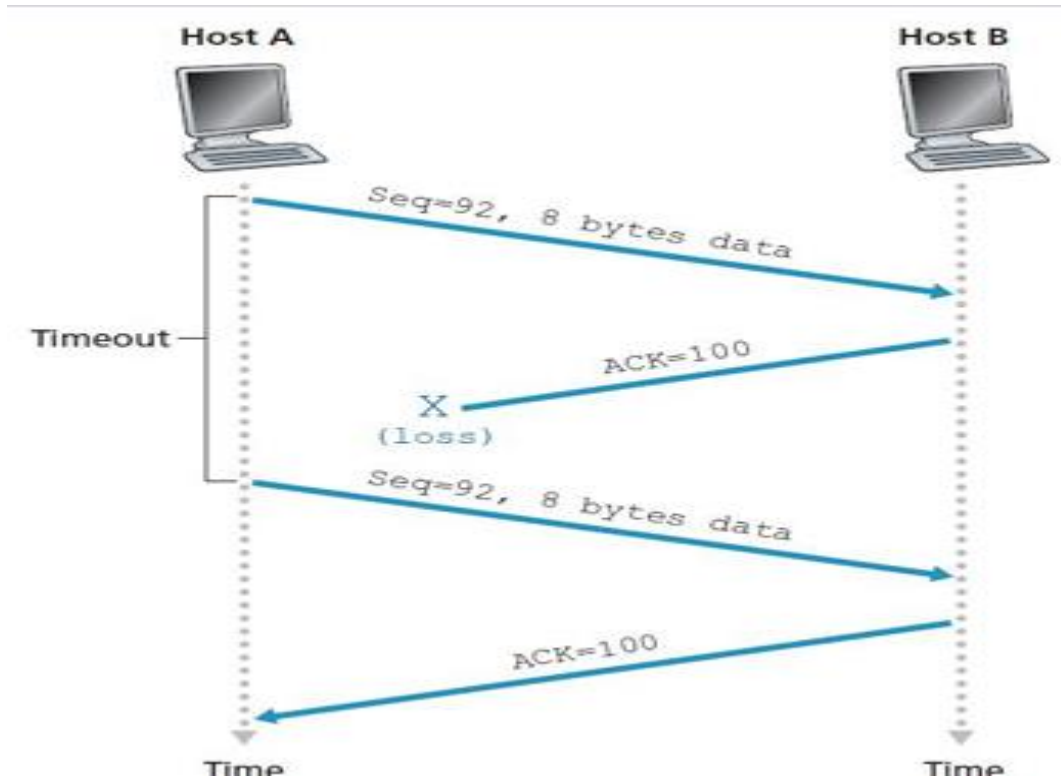
شرح مفهوم SendBase

SendBase هو نقطة البداية التي يعرف منها TCP أين وصل المستقبل في استلام البيانات.

إذا كان $SendBase = 501$ فهذا يعني أن آخر بايت تم استلامه بشكل صحيح هو 500.

يستخدم بروتوكول TCP إشعارات التأكيد التراكمية (cumulative acknowledgments) ، بحيث تؤكد القيمة y استلام جميع البايتات التي تسبق البايت رقم y. إذا كانت $y > SendBase$ فإن إشعار التأكيد (ACK) يؤكد استلام شريحة واحدة أو أكثر من الشرائح التي لم يتم تأكيدها سابقاً. وبناءً على ذلك، تقوم جهة الإرسال بتحديث متغير SendBase الخاص بها. كما تقوم أيضاً بإعادة تشغيل المؤقت إذا كان هناك حالياً أي شرائح لم يتم تأكيد استلامها بعد.

Reliable Data Transfer – Scenario 1 (Lost ACK)



في هذا السيناريو، يقوم Host A بإرسال Segment إلى Host B، ويستقبل Host B البيانات بشكل صحيح ويرسل ACK يؤكد استلامها، لكن المشكلة أن الـ ACK يضيع في الطريق ولا يصل إلى Host A.

ماذا يحدث عندما يضيع الـ ACK؟

بما أن Host A لم يستلم الـ ACK، فإنه يفترض أن الـ Segment لم يصل إلى Host B، وبالتالي ينتهي المؤقت (Timeout) فيقوم Host A بإعادة إرسال نفس الـ Segment مرة أخرى.

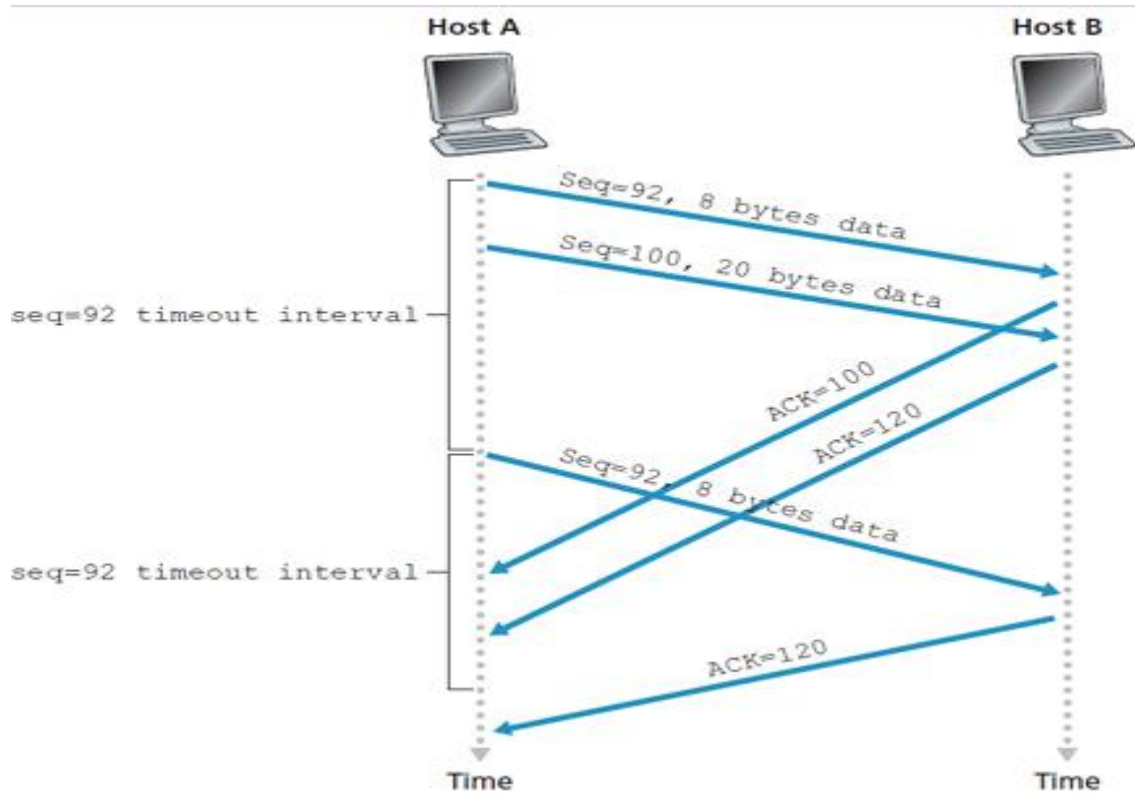
كيف يتصرف Host B عند استلام إعادة الإرسال؟

عندما يستقبل Host B الـ Segment المعاد إرساله ينظر إلى Sequence Number يلاحظ أن البيانات الموجودة في الـ Segment قد تم استلامها سابقاً لذلك يقوم TCP في Host B بتجاهل البيانات وعدم إضافتها مرة أخرى لكنه يعيد إرسال ACK لتأكيد الاستلام، بهذا الشكل TCP يمنع تكرار البيانات ويحافظ على تسلسل صحيح للبيانات.

ملاحظة: الـ Segment يبدأ من 92 ولكنه 8 bytes data ف $92 + 8 = 100$ لذلك الـ ACK= 100

Opportunities don't happen, you create them

Reliable Data Transfer – Scenario 2 (Two Segments, Lost ACKs)



في هذا السيناريو، يقوم Host A بإرسال مقطعين (Segments) متتاليين:

■ الأول يبدأ عند Seq = 92

■ الثاني يبدأ عند Seq = 100

كلاهما يصلان إلى Host B بدون أي مشاكل، ويقوم Host B بإرسال ACK لكل Segment لكن المشكلة هنا أن كلا ACKs لا يصلان إلى Host A قبل انتهاء المؤقت.

ماذا يحدث عند انتهاء المؤقت؟

عندما ينتهي المؤقت في Host A يفترض أن الـ Segment الأول لم يصل لذلك يقوم بإعادة إرسال Segment رقم 92 فقط ويعيد تشغيل المؤقت من جديد

لاحظ هنا TCP لا يعيد إرسال كل شيء دفعة واحدة، بل يبدأ دائماً بإعادة إرسال أقدم Segment غير مؤكد.

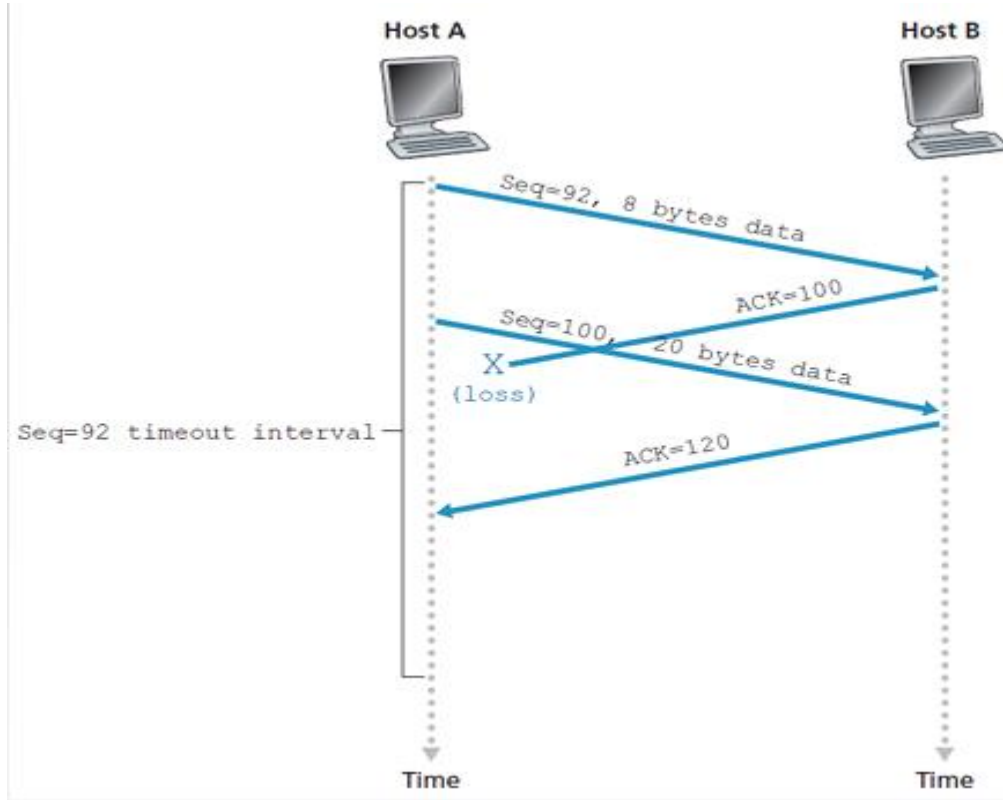
كيف يتصرف Host B عند استلام إعادة الإرسال؟

عندما يستقبل Host B الـ Segment المعاد إرساله يرى من Sequence Number أن البيانات الخاصة بـ Seq=92 قد وصلت سابقاً لذلك يقوم بتجاهل البيانات المكررة لكنه يرسل ACK جديد لتأكيد الاستلام

ماذا عن الـ Segment الثاني؟

هنا النقطة المهمة فإذا وصل ACK الخاص بالـ Segment الثاني (Seq=100) إلى Host A قبل انتهاء المؤقت الجديد فإن Host A لن يعيد إرسال Segment رقم 100 لأن ACK الذي وصل يكفي لتأكيد استلامه بهذا الشكل TCP يتجنب إعادة إرسال بيانات غير ضرورية، ويقلل الازدحام في الشبكة.

Reliable Data Transfer – Scenario 3 (Cumulative ACK Saves the Day)



في هذا السيناريو، يقوم Host A بإرسال مقطعين متتاليين:

▪ Segment الأول يبدأ عند Seq = 92

▪ Segment الثاني يبدأ عند Seq = 100

كلاهما يُرسل بشكل طبيعي. لكن المشكلة ضياع ACK الأول،

Host B يستقبل Segment الأول ويرسل ACK يؤكد استلامه، لكن ACK الخاص بال Segment الأول يضيع في الشبكة، Host A ، ينتظر وصول ACK والمؤقت يقترب من الانتهاء لكن قبل أن يحدث Timeout وقبل انتهاء المؤقت مباشرة، يستقبل Host A ACK = 120، وهذا يعني أن Host B يؤكد استلام كل البايتات حتى 119، أي أن ال Segment الأول (Seq=92, 8 bytes) و ال Segment الثاني (Seq=100, 20 bytes) كلاهما تم استلامهما بنجاح.

⚡ ماذا يفهم Host A من ACK = 120 ؟

Host A يدرك أن Host B استلم كل البيانات المرسلة و لا حاجة لإعادة إرسال أي Segment ، فيعاد ضبط المؤقت والاتصال يستمر بشكل طبيعي.

وهذه هي فائدة التأكيد التراكمي cumulative acknowledgments في TCP حيث ACK واحد يمكنه تأكيد استلام عدة Segments دفعة واحدة، حتى لو ضاعت ACKs السابقة.

Doubling the Timeout Interval _ مضاعفة زمن الانتظار في TCP

في الحالات الطبيعية، يحسب الـ TCP وقت المهلة (Timeout Interval) بناءً على متوسط وقت الرحلة الذهاب والإياب (EstimatedRTT) والتذبذب (DevRTT) لكن، ماذا لو حدث "انهيار" في الشبكة (Congestion Collapse)؟

إذا كانت الشبكة مزدحمة جداً لدرجة أن الحزم لا تصل، فمن المحتمل جداً أن الحزم التي أرسلها الـ TCP للتأكد "ستضيع" هي الأخرى بسبب الازدحام. إذا استمر الـ TCP في إعادة الإرسال بنفس الفترات الزمنية القصيرة، فإنه سيزيد الطين بلة ويغرق الشبكة المزدحمة بمزيد من الحزم، مما يؤدي لزيادة الازدحام أكثر.

لذلك عند حدوث Timeout :

- يعيد TCP إرسال أقدم Segment غير مؤكد
 - يضاعف قيمة TimeoutInterval بدلاً من حسابها من EstimatedRTT و DevRTT
 - ينتظر مدة أطول قبل إعادة الإرسال مرة أخرى
- هذا السلوك يسمى Exponential Backoff.

كيف تعمل الآلية؟

تخيل أن لديك حزمة لم تصل، والـ TimeoutInterval الحالي هو 0.75 ثانية:

1. المحاولة الأولى :

ينتهي الوقت (0.75 ثانية) ← لم يصل ACK ← يعيد الإرسال.

2. تعديل الوقت:

لا يحسب الـ TCP وقتاً جديداً بناءً على RTT ، بل يضاعف الوقت فوراً:

$$\text{NewTimeout} = 0.75 \times 2 = 1.5 \text{sec}$$

3. المحاولة الثانية: ينتهي الوقت (1.5 ثانية) ← لم يصل ACK ← يعيد الإرسال.

4. تعديل الوقت مجدداً :

يضاعف الوقت مرة أخرى:

$$\text{NewTimeout} = 1.5 \times 2 = 3.0 \text{sec}$$

وهكذا، يتضاعف الوقت بشكل أسّي (Exponentially) : $0.75 \leftarrow 1.5 \leftarrow 3.0 \leftarrow 6.0 \dots$

كيف يساعد هذا في التحكم بالازدحام؟

مضاعفة المؤقت يعطي الشبكة فرصة لتخفيف الازدحام ومعالجة التأخير واستعادة الاستقرار، وبالتالي يمنع TCP من إرسال المزيد من البيانات بسرعة قد تزيد المشكلة.

هذا السلوك يعتبر شكلاً بسيطاً من التحكم بالازدحام (Congestion Control) قبل ظهور خوارزميات أكثر تطوراً مثل:

- Slow Start
- Congestion Avoidance
- Fast Retransmit
- Fast Recovery

لتتعرف على Fast Retransmit إعادة الإرسال السريعة في TCP :

إعادة الإرسال المعتمدة على Timeout قد تكون بطيئة جداً لأن قيمة Timeout غالباً تكون كبيرة نسبياً وإذا ضاع Segment، سيضطر المرسل للانتظار حتى انتهاء المؤقت وهذا الانتظار يزيد زمن التأخير End-to-End Delay بشكل ملحوظ لذلك TCP يحتاج طريقة أسرع لاكتشاف فقدان Segment وهنا يأتي دور Fast Retransmit. بدلاً من انتظار انتهاء المؤقت، يستطيع المرسل اكتشاف فقدان Segment مبكراً من خلال Duplicate ACK

ماهو ال Duplicate ACK:

هو إشعار استلام (ACK) يعيد تأكيد استلام حزمة (Segment) سبق للمرسل أن تلقى تأكيداً لها بالفعل. بمعنى آخر: المستلم (Host B) يستمر في إرسال ACK لنفس البايتات (التي أكد استلامها مسبقاً بالفعل) لأنه ينتظر Segment مفقود لم يصل بعد.

كيف تكشف ال Duplicate ACKs عن فقدان Segment ؟

لنفرض أننا نرسل حزم متسلسلة: الحزمة رقم 1، ثم 2، ثم 3، ثم 4.

- 1. الحزمة رقم 2 فقدت في الطريق، لكن الحزم 1، 3، 4 وصلت بنجاح.
- 2. عند وصول الحزمة 1 يرسل المستلم ACK2 (أي: لقد استلمت 1، وأنتظر رقم 2).
- 3. عند وصول الحزمة 3 (وهي ليست التالية المطلوبة): المستلم لا يستطيع أن يقول "استلمت 3"، بل يجب عليه أن يطلب الحزمة التي كان ينتظرها. فيرسل ACK2 مرة أخرى. (هذا هو Duplicate ACK الأول).
- 4. عند وصول الحزمة 4 المستلم يرى أن هناك فجوة (لا يزال ينتظر 2)، فيرسل ACK2 مرة ثالثة. (هذا هو Duplicate ACK الثاني)

النتيجة: المرسل استلم ثلاث رسائل ACK2 هذا التكرار هو إنذار صريح بأن الحزمة رقم 2 ضاعت، وأن الحزم التي تليها (3 و 4) قد وصلت بالفعل.

TCP ACK Generation Recommendation [RFC 5681]

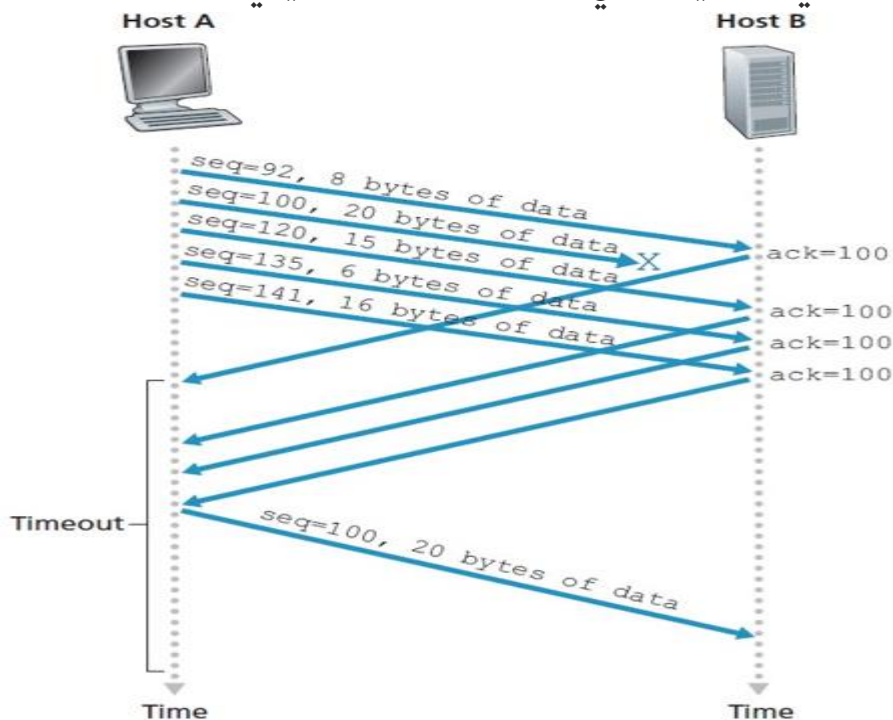
يقدم RFC 5681 توصيات حول كيفية قيام مستقبل TCP بإرسال ACKs بطريقة تقلل الازدحام وتحسن الأداء. تعتمد هذه التوصيات على حالة وصول ال segments وما إذا كانت مرتبة أو خارج الترتيب، وذلك لضمان كفاءة الإرسال وتقليل ال duplicate ACKs غير الضرورية.

الحدث Event	الإجراء المتخذ من مستقبل TCP (TCP Receiver Action)
وصول شريحة مرتبة تحمل رقم التتابع المتوقع. وجميع البيانات السابقة وصولاً إلى رقم التتابع المتوقع تم تأكيد استلامها بالفعل.	إشعار تأكيد مؤجل Delayed ACK: الانتظار لمدة تصل إلى 500 ميلي ثانية لوصول شريحة مرتبة أخرى. إذا لم تصل الشريحة التالية خلال هذا الفاصل الزمني، يتم إرسال ACK.
وصول شريحة مرتبة (in-order) تحمل رقم التتابع المتوقع. هناك شريحة مرتبة أخرى تنتظر بالفعل إرسال إشعار التأكيد الخاص بها.	إرسال إشعار تأكيد تراكمي واحد (single cumulative ACK) فوراً، ليؤكد استلام كلتا الشريحتين المرتبتين معاً.
وصول شريحة غير مرتبة (out-of-order) تحمل رقم تتابع أعلى من المتوقع. تم رصد فجوة (Gap detected).	إرسال إشعار تأكيد مكرر (duplicate ACK) فوراً، يشير إلى رقم التتابع للبايت المتوقع التالي (والذي يمثل الحد الأدنى من الفجوة).
وصول شريحة تملأ الفجوة جزئياً أو كلياً في البيانات المستلمة.	إرسال إشعار تأكيد (ACK) فوراً، بشرط أن تبدأ الشريحة عند الحد الأدنى من الفجوة.

مثال عن قاعدة Fast Retransmit

إذا استقبل المرسل ثلاثة Duplicate ACKs متتالية لنفس القيمة فإنه يقوم بالتالي:

1. تحديد الحزمة المفقودة: إذا كان الـ ACK المكرر هو ACK N، فإن الحزمة المفقودة هي الحزمة رقم N.
2. إعادة الإرسال الفوري: يقوم المرسل بإرسال الحزمة N فوراً.
3. تجاوز الـ Timer: الميزة هنا هي أنه يتم ذلك قبل انتهاء عداد الوقت (before the timer expires) هذا يوفر وقت الانتظار الذي كان سيحدث في حالة الـ Timeout التقليدي



1. المشكلة: ضياع شريحة في المنتصف:

المضيف أ (Host A) أرسل 5 شرائح من البيانات وراء بعضها مباشرة وبسرعة عالية:

1. الشريحة الأولى (seq=92) وصلت بسلام، فرد المضيف ب (Host B) بإشعار تأكيد يطلب الشريحة التالية :
ack=100.
2. الشريحة الثانية (seq=100) ضاعت في الشبكة.
3. الشرائح اللاحقة seq=120 و seq=135 و seq=141 وصلت كلها بسلام إلى المضيف ب.

2. رد فعل المستقبل (المضيف ب).

وفقاً لقواعد TCP (التي ذكرناها في جدول التوصيات السابق)، عندما تصل شريحة "غير مرتبة" وبينهما فجوة، يجب على المستقبل أن يرسل فوراً إشعار تأكيد مكرر (Duplicate ACK) يطلب فيه أول بايت مفقود.

- عندما وصلت الشريحة seq=120 للمستقبل، تذكر أنه لم يستلم البايت 100 بعد، فأرسل ack=100 (المكرر الأول)
- عندما وصلت الشريحة seq=135 أرسل مجدداً ack=100 (المكرر الثاني).
- عندما وصلت الشريحة seq=141 أرسل مجدداً ack=100 (المكرر الثالث).

3. قاعدة الـ 3 إشعارات مكررة (3 Duplicate ACKs)

عندما استقبل المضيف أ الإشعار الأصلي ack=100 متبوعاً بـ 3 إشعارات مكررة متتالية تطلب نفس الرقم (100) ، فهمت جهة الإرسال فوراً المعادلة التالية:

"بما أن المستقبل استلم 3 حزم تالية، ولكنه ما زال يطلب البايت رقم 100، فهذا يعني بالتأكيد أن الشريحة رقم 100 قد ضاعت في الطريق وليست مجرد متأخرة."

4. النتيجة (إعادة الإرسال السريع).

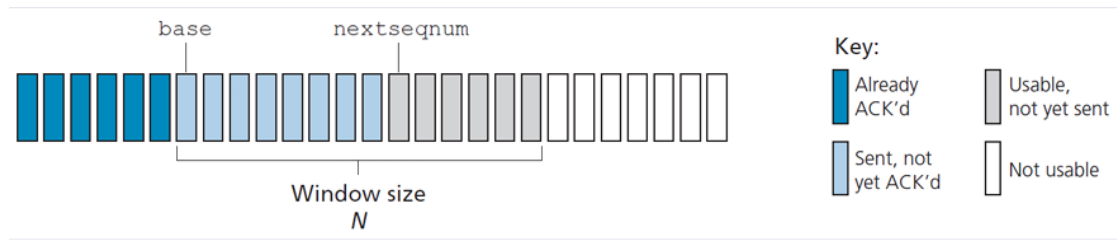
بدلاً من أن ينتظر المضيف أ انتهاء المؤقت الخاص بالشريحة 100 (والذي قد يستغرق وقتاً طويلاً بسبب بطء الاتصال كما يظهر في خط الـ Timeout)، قام المضيف أ بإعادة إرسال الشريحة المفقودة فوراً (seq=100) بمجرد وصول الإشعار المكرر الثالث.

Go-Back-N (GBN)

بروتوكول Go-Back-N هو أحد بروتوكولات ARQ – Automatic Repeat reQuest المستخدمة لضمان موثوقية نقل البيانات. يسمح للمرسل بإرسال عدة حزم (Packets) متتالية دون انتظار وصول ACK لكل واحدة منها. لكن هناك شرط مهم فلا يمكن للمرسل أن يرسل أكثر من N حزمة غير مؤكدة في نفس الوقت. القيمة N هي حجم نافذة الإرسال (Window Size) وهذه النافذة تحدد:

- أي الحزم يمكن إرسالها الآن

- أي الحزم تنتظر ACK
- أي الحزم لا يمكن إرسالها بعد



الصورة توضّح النافذة المنزلقة التي يستخدمها GBN :

- Packets تم تأكيدها (Already ACK'd) ← خارج النافذة
- Packets تم إرسالها ولم تُؤكّد بعد (Sent, not yet ACK'd) ← داخل النافذة
- Packets جاهزة للإرسال (Usable, not yet sent) ← داخل النافذة
- Packets غير قابلة للإرسال بعد (Not usable) ← خارج النافذة

والنافذة تتحرك (تنزلق) كلما وصل ACK جديد.

خطوات عمل GBN:

- المرسل يرسل حتى N حزمة متتالية دون انتظار ACK
- كل حزمة تحمل رقم تسلسل (Sequence Number)
- إذا وصل ACK لحزمة معينة، تتحرك النافذة للأمام.
- إذا ضاعت حزمة واحدة فقط، يقوم المستقبل بإرسال Duplicate ACKs للحزمة الأخيرة الصحيحة.
- عند حدوث Timeout أو اكتشاف فقدان حزمة يقوم المرسل بإعادة إرسال كل الحزم من الحزمة المفقودة فصاعداً.

لماذا يسمى Go-Back-N ؟

لأن المرسل يعود إلى الحزمة المفقودة ويعيد إرسال N حزمة بعدها، حتى لو كانت بعض هذه الحزم قد وصلت بالفعل للمستقبل. هذا يجعل البروتوكول بسيطاً لكنه قد يسبب إعادة إرسال غير ضرورية

ملاحظة: في Go-Back-N ، يمكن للمرسل إرسال عدة حزم دون انتظار ACK ، لكن لا يمكنه تجاوز حجم النافذة N عند فقدان حزمة، يعيد المرسل إرسال الحزمة المفقودة وكل الحزم بعدها.

ما هو base ؟

base هو: رقم التسلسل (Sequence Number) الخاص بأقدم Packet لم يصل ACK لها بعد. بمعنى آخر كل الحزم التي تحمل أرقام تسلسل أقل من base تم تأكيدها بالفعل، والمؤقت (Timer) في GBN مرتبط دائماً بالحزمة ذات الرقم base لأنها الأقدم غير المؤكدة.

ما هو nextseqnum ؟

nextseqnum هو: أصغر رقم تسلسل غير مستخدم بعد، أي رقم الحزمة التالية الجاهزة للإرسال.

يمثل الحد الأعلى للنافذة المنزلة، ويحدد آخر Packet يمكن للمرسل إرساله دون تجاوز حجم النافذة N .

تفسير النافذة المنزلة باستخدام nextseqnum base

1 المنطقة [0, base - 1]

هذه المنطقة تمثل Packets تم إرسالها وتم استلام ACK لها فلم تعد ضمن النافذة.

تظهر في الصورة باللون الأزرق الداكن (Already ACK'd)

2 المنطقة [base, nextseqnum - 1]

هذه المنطقة تمثل Packets تم إرسالها ولكن لم يصل ACK لها بعد وهي داخل النافذة.

تظهر في الصورة باللون الأزرق الفاتح (Sent, not yet ACK'd)

هذه المنطقة هي الجزء الحساس في GBN ، لأنها تحدد أين يقف المؤقت وما الذي يجب إعادة إرساله عند حدوث Timeout.

3 المنطقة [nextseqnum , base + N - 1]

تمثل هذه المنطقة الحزم التي يمكن إرسالها فوراً إذا وصل بيانات جديدة من الطبقة العليا (Upper Layer) .

هذه المنطقة هي الجزء "الجاهز للإرسال" داخل النافذة، وتظهر في الصورة باللون الأبيض الذي يمثل Usable, not (yet sent).

4 المنطقة غير القابلة للإرسال بعد

أي رقم التسلسل $base + N <$ لا يمكن استخدامه للإرسال حتى يتم تأكيد الحزمة ذات الرقم base.

لأن النافذة لا يمكن أن تتجاوز حجمها N والحزمة ذات الرقم base هي أقدم حزمة غير مؤكدة فلا يمكن للنافذة أن تتحرك للأمام إلا عند وصول ACK لهذه الحزمة، لذلك تبقى الأرقام الأكبر من $base + N$ "غير قابلة للاستخدام" تظهر باللون الرمادي (Not usable)

كيف تنزلق النافذة؟

عندما يصل ACK للحزمة ذات الرقم base:

- تتحرك النافذة خطوة للأمام
- يصبح $base = base + 1$
- يصبح nextseqnum قادراً على التقدم أيضاً
- تفتح النافذة مساحة جديدة لإرسال حزم إضافية

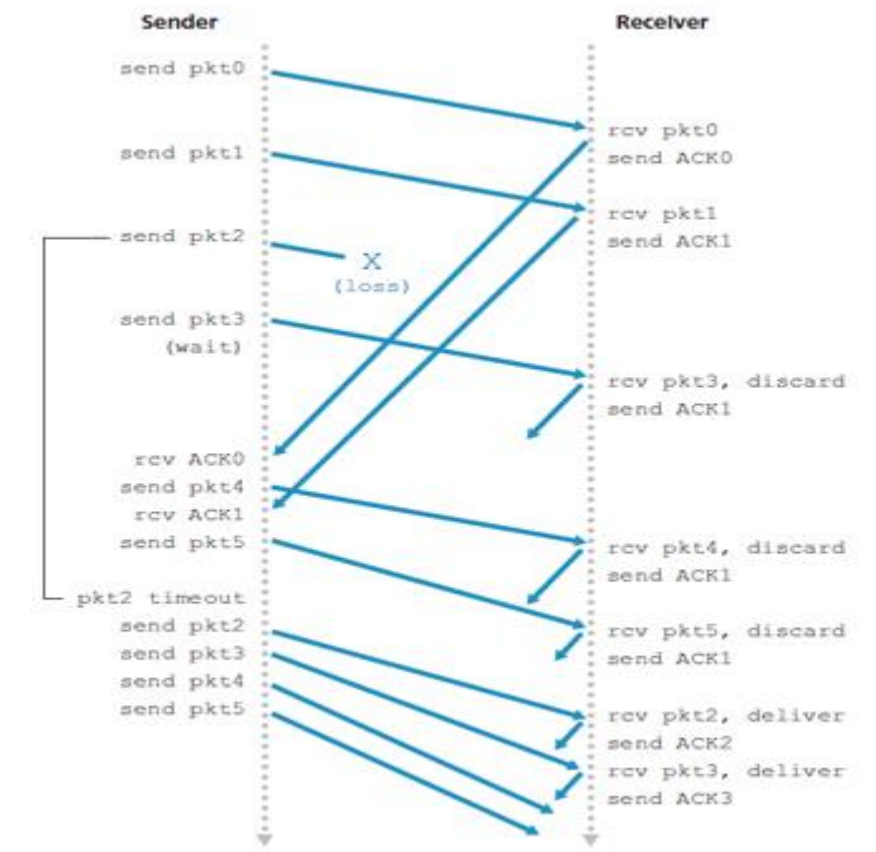
ولهذا السبب يسمى البروتوكول Sliding Window Protocol.

لماذا نحتاج نافذة؟ (Flow Control)

السبب الأساسي لوجود النافذة هو التحكم في التدفق Flow Control فالنافذة تمنع المرسل من إرسال عدد كبير جداً من الحزم دفعة واحدة وإغراق المستقبل ببيانات لا يستطيع معالجتها والتسبب في ازدحام أو فقدان حزم، بالتالي النافذة تضمن أن المرسل يرسل البيانات بمعدل يستطيع المستقبل التعامل معه.



Go-Back-N in Operation – Example



في هذا المثال، يقوم المرسل Sender بإرسال سلسلة من الحزم (pkt0, pkt1, pkt2, ...) إلى المستقبل Receiver كل حزمة تحمل رقم تسلسل، ويقوم المستقبل بإرسال ACK عند استلامها. لكن يحدث فقدان لحزمة واحدة، وهذا يكشف لنا كيف يتصرف بروتوكول GBN :

(1) إرسال الحزم بالترتيب

المرسل يرسل pkt0, pkt1, pkt2 وكلها ترسل دون انتظار ACK ، لأن GBN يسمح بإرسال عدة حزم داخل النافذة.

(2) استلام الحزم الصحيحة

المستقبل يستقبل:

■ pkt0 ← يرسل ACK0

■ pkt1 ← يرسل ACK1

(3) حدوث فقدان للحزمة pkt2

الحزمة pkt2 تضيع في الطريق (loss) والمستقبل لا يستلمها، وبالتالي لا يستطيع تسليم pkt3 أو pkt4 أو pkt5 للتطبيق لأن GBN يعتمد على in-order delivery لذلك يقوم المستقبل بتجاهل أي حزمة تأتي بعد pkt2

(4) انتهاء المؤقت عند المرسل

المرسل ينتظر وصول ACK2 لكنه لا يصل فعندما ينتهي المؤقت (Timeout) يعيد المرسل إرسال pkt2 ثم يعيد إرسال كل الحزم بعد pkt2 (pkt3, pkt4, pkt5)

(5) استلام pkt2

المستقبل يستقبل pkt2 فيسلمها للتطبيق ويرسل ACK2

الآن يستطيع المستقبل قبول الحزم التالية:

■ rcv pkt3 → deliver → ACK3

■ rcv pkt4 → deliver → ACK4

■ rcv pkt5 → deliver → ACK5

وبهذا يعود الاتصال للعمل بشكل طبيعي.

الخلاصة

في GBN إذا ضاعت حزمة واحدة، يقوم المستقبل بتجاهل كل الحزم التالية لها، وعند حدوث Timeout ، يعيد المرسل إرسال الحزمة المفقودة وكل الحزم بعدها. هذا يضمن التسليم بالترتيب لكنه قد يسبب إعادة إرسال غير ضرورية.

Selective Repeat (SR) Protocol

يُعد بروتوكول Selective Repeat (SR) نسخة أكثر كفاءة من Go-Back-N ، إذ يتجنب إعادة إرسال جميع الحزم بعد الحزمة المفقودة، ويكتفي بإعادة إرسال الحزم التي فُقدت أو تضررت فقط، مما يقلل بشكل كبير من استهلاك النطاق الترددي. ولتحقيق ذلك، يعتمد SR على إقرار فردي (Individual ACK) لكل حزمة تصل بشكل صحيح، بحيث يعرف المرسل بالضبط أي الحزم وصلت وأيها تحتاج إلى إعادة إرسال. وكما في GBN يستخدم SR نافذة إرسال بحجم N ، لكن الاختلاف هو أن نافذة SR قد تحتوي على حزم تم تأكيدها وأخرى لم تُؤكّد بعد، بينما نافذة GBN تضم فقط الحزم غير المؤكدة. أما من حيث السلوك، فإن GBN يعيد إرسال جميع الحزم بعد الحزمة المفقودة ويتجاهل المستقبل الحزم خارج الترتيب ويستخدم ACK تراكمي، في حين أن SR يعيد فقط الحزمة المفقودة، ويقبل المستقبل الحزم خارج الترتيب ويخزنها، ويرسل ACK منفصل لكل حزمة، مما يجعل البروتوكول أكثر كفاءة ومرونة في التعامل مع فقدان الحزم.

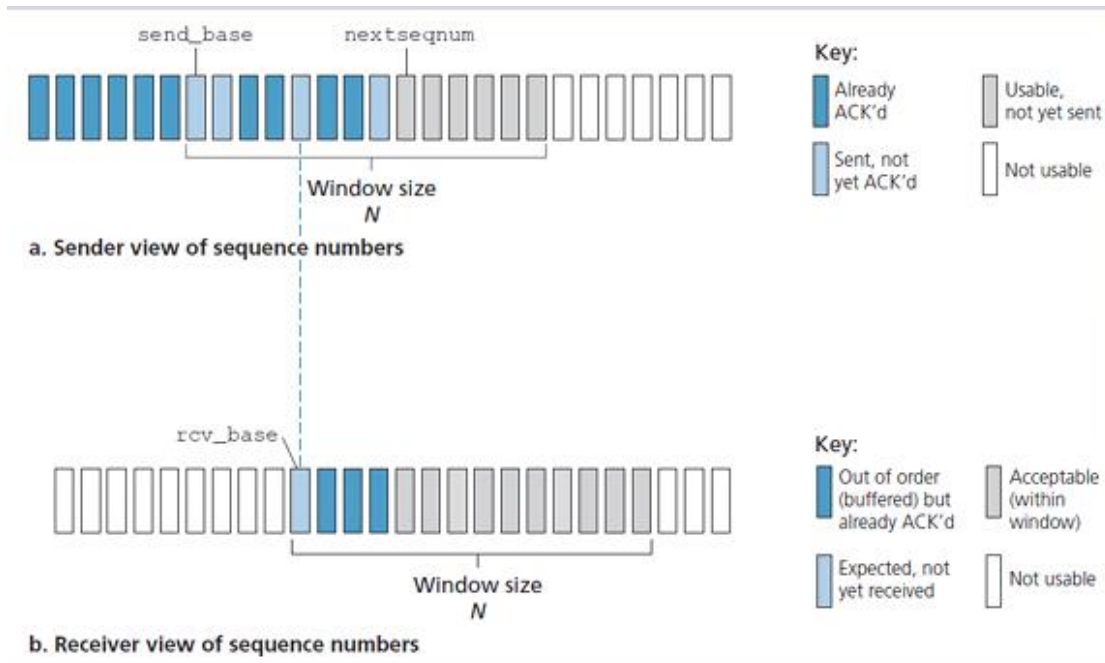
Selective Repeat (SR) – Sender & Receiver Views

في SR ، المرسل والمستقبل يمتلكان نافذتين مختلفتين:

■ نافذة المرسل تحدد الحزم التي يمكن إرسالها أو تنتظر ACK

■ نافذة المستقبل تحدد الحزم التي يمكن قبولها أو تخزينها مؤقتاً

فالمستقبل في SR يتصرف بذلك، فإذا وصلت حزمة صحيحة، فإنه يرسل ACK لها حتى لو كانت خارج الترتيب. فعندما تصل حزمة خارج الترتيب المستقبل يخزنها في Buffer و ينتظر وصول الحزمة ذات الرقم الأصغر (التي تسبقها)، وعندما تصل الحزمة المفقودة، يقوم المستقبل بتسليم دفعة كاملة من الحزم بالترتيب إلى الطبقة العليا



Selective Repeat (SR) – Sender & Receiver Events and Actions

أحداث المرسل وتصرفاته – SR Sender

1 وصول بيانات من الطبقة العليا (Data received from above)

عندما يستقبل المرسل بيانات جديدة:

- يفحص رقم التسلسل التالي المتاح `nextseqnum`
- إذا كان رقم التسلسل داخل نافذة الإرسال ← يقوم بتغليف البيانات في Packet ويرسلها
- إذا كان رقم التسلسل خارج النافذة ←
 - إما يخزن البيانات في Buffer
 - أو يعيدها للطبقة العليا لإرسالها لاحقاً (كما في GBN)

2 حدوث Timeout

في SR كل Packet يمتلك مؤقتاً خاصاً به فإذا انتهى مؤقت Packet معين يعيد المرسل إرسال هذا الـ Packet فقط ولا يعيد إرسال كل الحزم كما في GBN

3 وصول ACK

عندما يصل ACK يعلم المرسل أن Packet معين قد تم استلامه بنجاح:

- إذا كان رقم التسلسل للـ Packet هو `send_base`

- يحرك المرسل نافذة الإرسال للأمام
- يحدد `send_base` الجديد كأصغر رقم تسلسل غير مؤكد

- إذا تحركت النافذة للأمام

- يتم إرسال أي Packet غير مرسل سابقاً وأصبح الآن داخل النافذة

أحداث المستقبل وتصرفاته – SR Receiver

1 وصول Packet داخل نافذة المستقبل [rcv_base , rcv_base + N - 1]

إذا وصل Packet صحيح داخل النافذة يرسل المستقبل Selective ACK لهذا الـ Packet :

- إذا لم يكن المستقبل قد استلم هذا Packet سابقاً ← يخزنه في Buffer

- إذا كان رقم التسلسل للـ Packet هو `rcv_base` أي مساوياً لقاعدة نافذة الاستقبال حينها:

1. يقوم المستقبل بتسليم هذا Packet للطبقة العليا
2. ثم يسلم أي Packets مخزنة متتالية بعده
3. يحرك نافذة المستقبل للأمام بعدد الحزم التي تم تسليمها

2 وصول Packet قد تم استلامه سابقاً [rcv_base - N, rcv_base - 1]

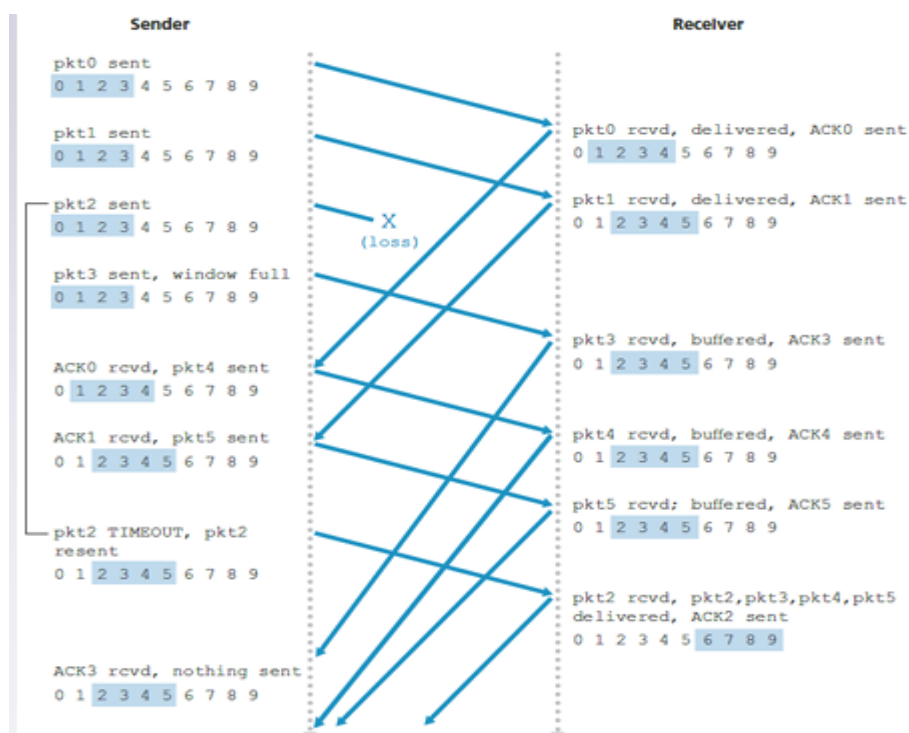
إذا وصل Packet سبق استلامه يجب على المستقبل إرسال ACK مرة أخرى حتى لو كان ACK قد أرسل سابقاً هذا يمنع المرسل من إعادة إرسال Packet غير مفقود

3 في أي حالة أخرى (Otherwise) :

يتم تجاهل الحزمة.



Selective Repeat (SR) – Example in Operation



في هذا المثال، يرسل المرسل مجموعة من الحزم المتتالية pkt0، pkt1، pkt2، pkt3، pkt4، pkt5. يستقبل المستقبل بعض الحزم بشكل صحيح، بينما تضيع إحدى الحزم في الطريق، وهنا يظهر الفرق بين SR و Go-Back-N

1. إرسال الحزم بالترتيب

يقوم المرسل بإرسال الحزم واحدة تلو الأخرى دون انتظار ACK لكل حزمة، لأن نافذة SR تسمح بوجود عدة حزم غير مؤكدة في نفس الوقت.

pkt0 تصل وتسلم ← المستقبل يرسل ACK0

pkt1 تصل وتسلم ← المستقبل يرسل ACK1

2. فقدان الحزمة pkt2

الحزمة pkt2 تضيع في الطريق ولا تصل إلى المستقبل وهنا يبدأ الاختلاف بين SR و GBN:

في SR المستقبل لا يتجاهل الحزم التالية:

■ يستقبل pkt3 يخزنها في الـ buffer ويرسل ACK3

■ يستقبل pkt4 يخزنها ويرسل ACK4

■ يستقبل pkt5 يخزنها ويرسل ACK5

المستقبل ينتظر فقط وصول pkt2 ليتمكن من تسليم الحزم المخزنة بالترتيب.

3. انتهاء مؤقت pkt2 عند المرسل

المرسل ينتظر ACK2 لكنه لا يصل فعندما ينتهي المؤقت يعيد المرسل إرسال pkt2 فقط ولا يعيد إرسال pkt3 أو pkt4 أو pkt5

4. وصول pkt2 إلى المستقبل

عندما تصل pkt2 يقوم المستقبل بتسليم pkt2 للطبقة العليا ثم يقوم بتسليم الحزم المخزنة بالترتيب pkt3 ← pkt4 ← pkt5 ويرسل ACK2 لتأكيد وصول الحزمة المفقودة بهذا الشكل، يتم استعادة الترتيب الصحيح دون إعادة إرسال غير ضرورية.

هل TCP هو Go-Back-N أم Selective Repeat؟

يُصنف بروتوكول TCP كبروتوكول هجين (Hybrid) يجمع بين خصائص بروتوكولي Go-Back-N (GBN) و Selective Repeat (SR)، حيث يشبه GBN في اعتماده على الإقرارات التراكمية (Cumulative ACKs) واكتفائه بتتبع مؤشرين أساسيين للبيانات المرسل:

■ SendBase = أصغر رقم تسلسل غير مؤكد

■ NextSeqNum = رقم البايت التالي للإرسال

ومع ذلك، يتفوق TCP عليه من خلال قدرته على تخزين الحزم المستلمة خارج الترتيب (Out-of-order) buffering مما يمنعه من إعادة إرسال السلسلة كاملة في حال فقدان إقرار واحد كما يفعل GBN

مثال للتوضيح:

المرسل يرسل الحزم: 1، 2، ...، N كل الحزم تصل للمستقبل بدون خطأ لكن ACK الخاص بالحزمة n يضع حيث $n < N$ وباقي ACKs تصل للمرسل قبل انتهاء المؤقت ماذا يحدث؟
في GBN:

يعيد المرسل الحزمة n وكل الحزم بعدها $n+1$ ، $n+2$ ، ...، N حتى لو كانت قد وصلت بالفعل!
في TCP:

يعيد المرسل الحزمة n فقط وحتى هذه الحزمة لن يعيدها إذا وصل ACK للحزمة $n+1$ قبل انتهاء المؤقت الخاص ب n وهذا سلوك أقرب ل Selective Repeat.

وتكتمل ملامح البروتوكول لتتقرب من نظام Selective Repeat عند تفعيل ميزة إقرار الاستلام الانتقائي (SACK) Selective Acknowledgment، والتي تسمح لمستقبل TCP إمكانية تأكيد استلام القطع التي وصلت خارج الترتيب بشكل انتقائي، بدلاً من الاكتفاء بالإقرار التراكمي لآخر قطعة تم استقبالها بشكل صحيح ومرتب. عند دمج هذا التعديل مع إعادة الإرسال الانتقائي (أي تخطي إعادة إرسال القطع التي تم تأكيد استلامها بشكل انتقائي من قبل المستقبل) يبدو بروتوكول TCP شبيهاً جداً ببروتوكول SR العام لدينا. وبالتالي، وكما ذكرنا فمن الأفضل تصنيف آلية معالجة الأخطاء في TCP على أنها مزيج (هجين) بين بروتوكولات SR وGBN.

Flow Control

- عند استقبال بايتات صحيحة ومتسلسلة، يقوم بروتوكول TCP بوضعها في مخزن الاستقبال (ReceiveBuffer) ويقوم التطبيق (Application Process) بقراءة البيانات من هذا المخزن، ولكن ليس بالضرورة فور وصولها. ولذلك إذا كان التطبيق بطيئاً في القراءة، فقد يقوم المرسل بإرسال كميات كبيرة من البيانات بسرعة تفوق قدرة الاستيعاب والنتيجة حدوث فيضان لمخزن الاستقبال (Buffer Overflow)، مما يؤدي لفقدان البيانات.

إن TCP يوفر خدمة التحكم في التدفق (Flow Control) لمنع هذا الفيضان وتعرف هذه الخدمة بأنها خدمة مطابقة السرعات (Speed-matching service) أي مطابقة معدل إرسال المرسل مع معدل قراءة التطبيق لدى المستقبل.

آلية التنفيذ (Implementation Mechanism)

نافذة الاستقبال: (Receive Window)



يستخدم TCP متغيراً يسمى نافذة الاستقبال (Receive Window) لإدارة هذه العملية. وظيفتها: إعطاء المرسل فكرة عن حجم المساحة الفارغة المتاحة في مخزن الاستقبال لدى الطرف الآخر. بما أن اتصال TCP هو ثنائي الاتجاه (Full-duplex)، فإن كل طرف في الاتصال يحتفظ بنافذة استقبال مستقلة خاصة به.

مثال:

لنفترض أن المضيف A يرسل ملفاً ضخماً إلى المضيف B، فيخصص المضيف B مساحة للمخزن تسمى RcvBuffer. نعرف المتغيرات التالية:

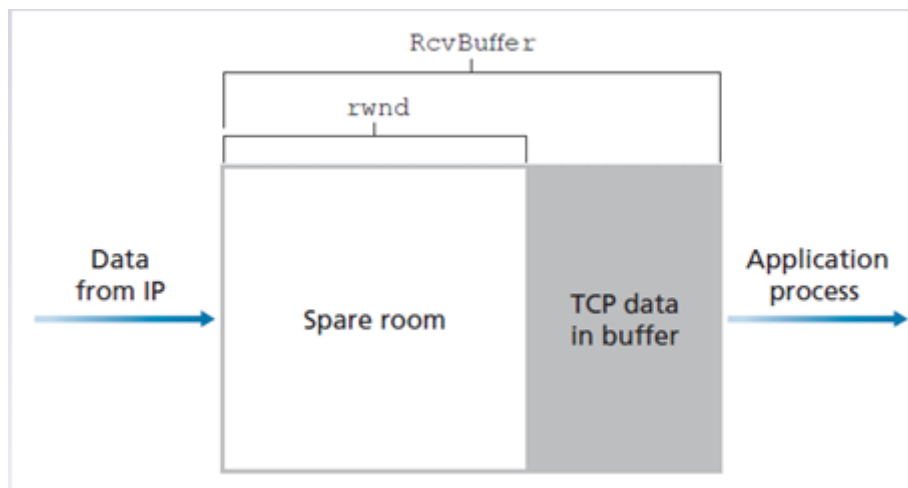
LastByteRead: آخر بايت قرأه التطبيق من buffer-

LastByteRcvd: آخر بايت وصل من الشبكة وتم وضعه في buffer-

لضمان عدم تجاوز سعة المخزن المخصصة، يجب دائماً تحقيق الشرط التالي:

$$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$$

TCP Flow Control – Receive Window (rwnd)



ما هو rwnd ؟

الـ receive window (rwnd) هو القيمة التي يخبر بها المستقبل المرسل: "هذا هو مقدار المساحة المتاحة لدي الآن في buffer"، وهذه القيمة تتغير باستمرار لأنها تعتمد على كمية البيانات التي وصلت من الشبكة وكمية البيانات التي قرأها التطبيق من buffer. ويحسب بالمعادلة التالية:

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

حيث:

- RcvBuffer: حجم الـ buffer المخصص للاتصال
- LastByteRcvd: آخر بايت وصل من الشبكة وتم وضعه في الـ buffer

▪ LastByteRead: آخر بايت قرأه التطبيق من الـ buffer

والفرق $LastByteRcvd - LastByteRead$ يمثل عدد البايتات الموجودة حالياً داخل الـ buffer وبالتالي:

▪ إذا كان الفرق كبيراً ← المساحة المتاحة قليلة ← rwnd صغير

▪ إذا كان الفرق صغيراً ← المساحة المتاحة كبيرة ← rwnd كبير

لماذا rwnd متغير (Dynamic) ؟

لأن التطبيق يقرأ البيانات من الـ buffer بشكل غير منتظم والبيانات تصل من الشبكة بشكل مستمر لذلك المساحة المتاحة تتغير لحظة بلحظة وبالتالي rwnd يجب أن يتغير باستمرار و نقول أنه قيمة ديناميكية

كيف يخبر Host B المرسل Host A بقيمة rwnd ؟

كل Segment يرسله Host B إلى Host A يحتوي على قيمة rwnd الحالية داخل حقل Receive Window في الـ TCP header وبهذا الشكل المرسل يعرف دائماً مقدار المساحة المتاحة ويضبط سرعة الإرسال بناءً على ذلك فلا يحدث buffer Overflow

ملاحظة:

عند بداية الاتصال $rwnd = RcvBuffer$ لأن الـ buffer يكون فارغاً تماماً وكما ذكرنا سابقاً المرسل يحافظ على متغيرين مهمين:

LastByteSent آخر بايت أرسله المرسل إلى المستقبل

LastByteAcked آخر بايت تم تأكيده (ACK) من المستقبل

والفرق بينهما $LastByteSent - LastByteAcked$ يمثل كمية البيانات غير المؤكدة الموجودة حالياً في الشبكة ولكي لا يملأ المرسل الـ buffer الخاص بالمستقبل، يجب أن يضمن دائماً:

$$LastByteSent - LastByteAcked \leq rwnd$$

المشكلة: كيف يحدث الانسداد (The Block) ؟

لنتخيل السيناريو التالي:

1. امتلاء المخزن: المضيف (ب) استقبل بيانات كثيرة ولم يقرأها التطبيق بسرعة، فامتأل المخزن المؤقت لديه تماماً.
2. إرسال التحذير: قام المضيف (ب) بإرسال إقرار استلام (ACK) للمضيف (أ) يخبره فيه بأن النافذة أصبحت صفراً ($rwnd = 0$). هذا يعني "توقف عن الإرسال فوراً، ليس لدي مساحة".
3. الصمت التام: المضيف (أ) التزم بالتعليمات وتوقف تماماً عن الإرسال و ينتظر إشارة جديدة. في هذه الأثناء، المضيف (ب) ليس لديه أي بيانات يرسلها إلى (أ).

هنا يكمن الخلل: بعد فترة قام التطبيق في المضيف (ب) بقراءة البيانات وتفرغ المخزن المؤقت، فأصبحت هناك مساحة واسعة لاستقبال البيانات. لكن بروتوكول TCP في المضيف (ب) لا يرسل تحديثاً تلقائياً ليقول "لقد فرغت المساحة"، لأن بروتوكول TCP مصمم ألا يرسل أي قطعة (Segment) إلا في حالتين فقط:

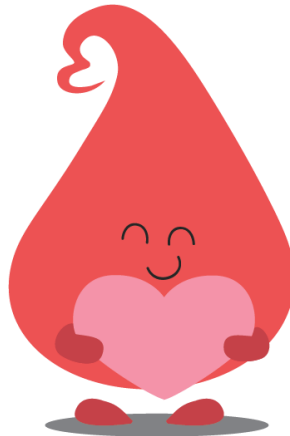
- إذا كان لديه بيانات جديدة يود إرسالها.
 - أو إذا كان يرد على بيانات وصلته بإقرار استلام (ACK).
- النتيجة**: المضيف (أ) ينتظر (ب) لفتح النافذة، والمضيف (ب) ينتظر (أ) ليرسل شيئاً ليرد عليه! يدخل الطرفان في حالة جمود تام (Deadlock) ويتوقف الاتصال نهائياً.

الحل: "قطع الاستطلاع (Window Probing)"

لحسم هذا الجمود، وضعت معايير TCP قاعدة إجبارية للمرسل (المضيف أ):
 "حتى لو أخبرك المستقبل أن النافذة صفر، لا تتوقف تماماً. استمر في إرسال قطع صغيرة جداً تحتوي على (1 بايت) فقط، من البيانات بشكل دوري".

كيف يحل هذا البايت الواحد المشكلة؟

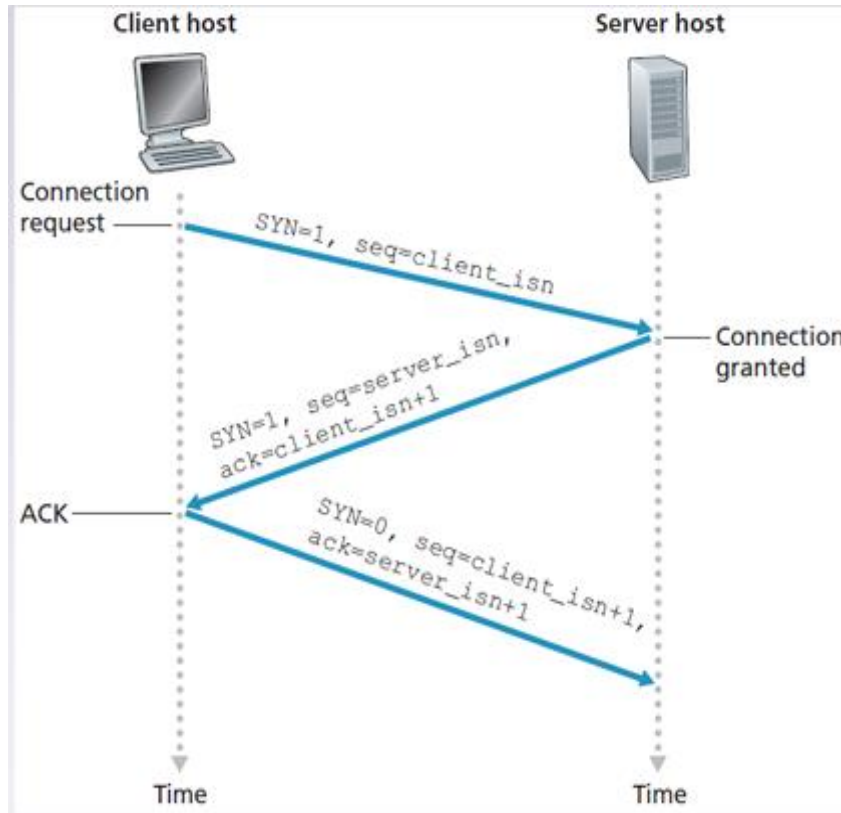
1. عندما يرسل المضيف (أ) هذه القطعة الصغيرة (التي تسمى Window Probe)، فإنها تجبر المضيف (ب) على الرد بإقرار استلام (ACK).
2. عندما يرسل المضيف (ب) هذا الرد، فإنه يُدرج فيه تلقائياً القيمة المحدثة لنافذة الاستقبال (rwnd).
3. إذا كان المخزن لا يزال ممتلئاً، سيعود الرد بـ (rwnd = 0)، فينتظر (أ) قليلاً ثم يعيد المحاولة بـ 1 بايت آخر.
4. بمجرد أن يتفرغ المخزن في (ب)، سيعود الرد بقيمة أكبر من الصفر (مثلاً rwnd = 1000)، وهنا يعرف المضيف (أ) أن الطريق أصبح مفتوحاً ويعود لإرسال البيانات.



TCP Connection Management

لنفترض أن هناك عملية (process) تعمل في أحد المضيفين (العميل client) تريد بدء اتصال مع عملية أخرى في مضيف آخر (الخادم server) تقوم بعملية تطبيق العميل أولاً بإعلام بروتوكول TCP الخاص بالعميل برغبتها في إنشاء

اتصال مع عملية في الخادم. بعد ذلك، يبدأ بروتوكول TCP في جانب العميل بإنشاء اتصال TCP مع بروتوكول TCP في جانب الخادم بالطريقة التالية:



Step 1 — SYN: طلب الاتصال من العميل

عندما يطلب التطبيق في الـ Client إنشاء اتصال:

1. يخبر الـ Client TCP بأنه يريد الاتصال بالـ Server
2. يقوم TCP في الـ Client بإرسال Segment خاص إلى الـ Server
3. هذا الـ Segment :

- لا يحتوي بيانات تطبيق
 - يحتوي SYN = 1 (SYN) : هو أحد بتات الراية (flag bits) في ترويسة القطعة (header)
 - يسمى SYN Segment
- يختار العميل رقم تسلسل ابتدائي client_isn بشكل عشوائي و يضعه في حقل Sequence Number

4. يتم تغليف هذا الـ Segment داخل IP Datagram وإرساله إلى الـ Server

Step 2 — SYNACK: ردّ الخادم على طلب الاتصال

بعد أن يصل الـ SYN Segment من العميل إلى الخادم:

1. الخادم يستخرج الـ TCP SYN segment من الـ IP datagram

2. يخصص الخادم الموارد اللازمة للاتصال الجديد:

- TCP buffers
- المتغيرات الخاصة بالاتصال

3. ثم يرسل للخادم Connection-granted segment يحتوي على ثلاث معلومات مهمة:

↪ أولاً: $SYN = 1$

الخادم يعلن أنه يوافق على إنشاء الاتصال.

↪ ثانياً: $ACK = client_isn + 1$

الخادم يؤكد استلام SYN من العميل.

↪ ثالثاً: $server_isn$

الخادم يختار رقم تسلسل ابتدائي خاص به ويضعه في حقل Sequence Number .

ملاحظة: هذا الـ Segment يُسمّى SYNACK Segment ولا يحتوي بيانات من طبقة التطبيقات، لكنه يحتوي معلومات التحكم الأساسية لإنشاء الاتصال.

ACK — Step 3: تأكيد العميل على ردّ الخادم

عندما يستقبل العميل SYNACK، يخصص العميل أيضاً buffers والمتغيرات الخاصة بالاتصال ويرسل Segment جديد إلى الخادم يحتوي على:

$$ACK = server_isn + 1$$

لتأكيد استلام SYNACK من الخادم.

$$SYN = 0$$

لأن الاتصال أصبح الآن مُنشأً بالكامل.

ملاحظة:

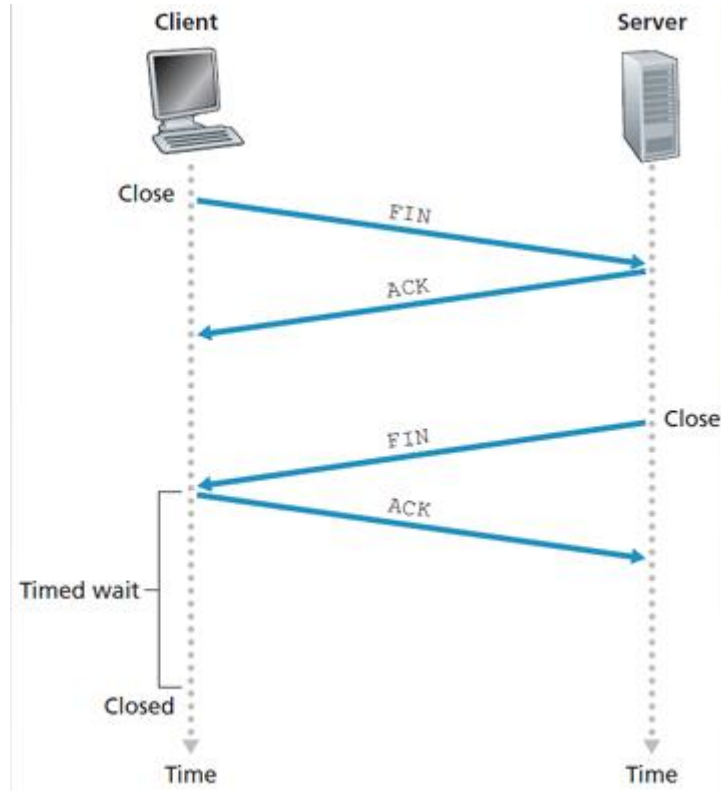
هذا الـ ACK الأخير قد يحتوي بيانات تطبيق (client-to-server data)، وهذا يسمح للعميل ببدء إرسال البيانات مباشرة بعد إنشاء الاتصال.

بعد الخطوات الثلاث

بعد اكتمال:

SYN
SYNACK

يصبح الاتصال مفتوحاً وجاهزاً لتبادل البيانات ، كل ال Segments القادمة سيكون فيها SYN = 0
تسمى هذه العملية: Three-Way Handshake لأنها تتكون من ثلاث رسائل بين العميل والخادم.



ولكن من يمكنه إنهاء الاتصال؟

أي طرف في الاتصال يمكنه إنهاءه العميل Client أو الخادم Server
فعند الإنهاء يتم تحرير الموارد (buffers + variables) في كلا الطرفين.

مثال: العميل يريد إنهاء الاتصال

Step 1 — العميل يرسل FIN

عندما يقرر التطبيق في العميل إغلاق الاتصال يصدر التطبيق أمر close فيقوم TCP في العميل بإرسال Segment خاص إلى الخادم وهذا ال Segment يحتوي على FIN = 1 (بت راية (flag bit) في ترويسة القطعة) ولكنه لا يحتوي بيانات تطبيق غالباً

هذا ال Segment يعني: "انتهيت من إرسال البيانات، أريد إغلاق الاتصال"

Step 2 — الخادم يرسل ACK

عندما يستقبل الخادم الـ FIN يرسل ACK للعميل وهذا يؤكد استلام طلب الإنهاء لكن الخادم قد لا يكون جاهزاً للإغلاق بعد (قد يكون لديه بيانات متبقية لإرسالها)

Step 3 — الخادم يرسل FIN

بعد أن ينتهي الخادم من إرسال بياناته يرسل Segment يحتوي $FIN = 1$ وهذا يعني: "انتهيت أنا أيضًا من إرسال البيانات، يمكننا إغلاق الاتصال"

Step 4 — العميل يرسل ACK

العميل يستقبل FIN من الخادم فيرسل ACK أخيراً وبعد ذلك يدخل العميل في حالة Timed Wait لفترة قصيرة (هذه الفترة تمنع فقدان ACK الأخير وظهور Segments متأخرة من اتصال قديم).
بعد انتهاء Timed Wait الاتصال يُغلق بالكامل، ويتم تحرير الموارد في كلا الطرفين.

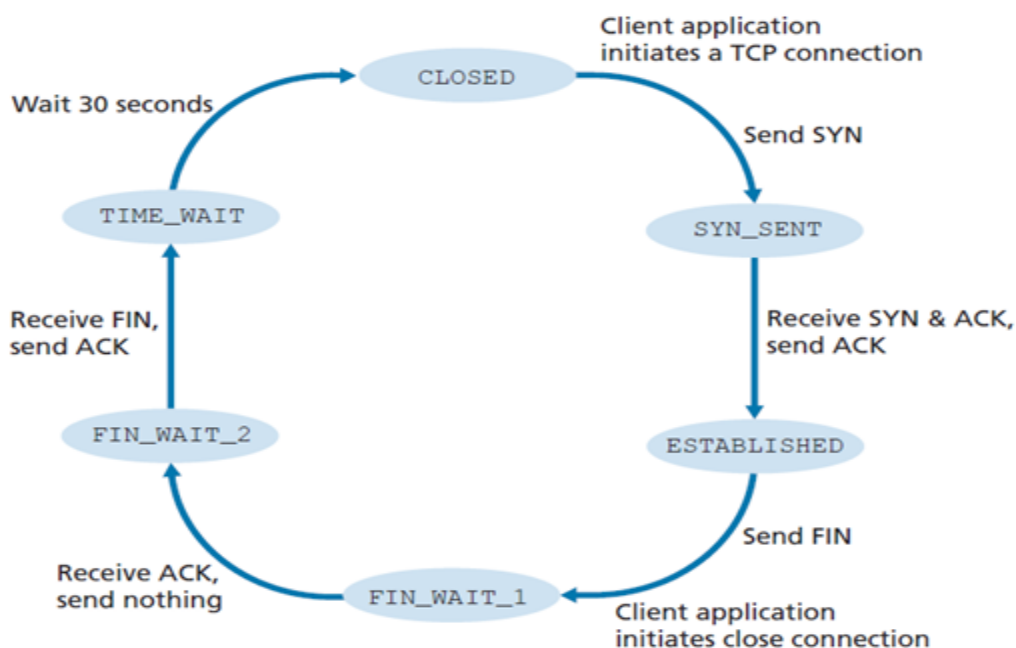
لماذا الإنهاء يحتاج 4 رسائل؟

لأن كل طرف يجب أن يعلن أنه انتهى من إرسال البيانات (FIN) ويؤكد استلام إعلان الطرف الآخر (ACK) وبذلك يصبح لدينا:

- FIN من العميل
- ACK من الخادم
- FIN من الخادم
- ACK من العميل

ولهذا يسمى: Four-Way Handshake

A Typical Sequence of TCP States Visited by a Client TCP



عندما يقوم العميل بإنشاء اتصال TCP ثم إغلاقه، فإنه يمرّ بمجموعة من الحالات (States) وفقاً لـ TCP FSM (Finite State Machine) هذه الحالات تضمن أن الاتصال يتم بشكل موثوق ومنظم.

1. CLOSED

هذه هي الحالة الابتدائية حيث لا يوجد اتصال، لا توجد موارد مخصصة، ينتظر العميل أن يطلب التطبيق إنشاء اتصال

2. SYN_SENT

عندما يطلب التطبيق إنشاء اتصال:

- يرسل العميل SYN إلى الخادم
- ينتقل إلى حالة SYN_SENT
- ينتظر وصول SYN + ACK من الخادم

هذه هي خطوة Step 1 في الـ Three-Way Handshake .

3. ESTABLISHED

عندما يستقبل العميل:

- SYN + ACK من الخادم
- يرسل العميل ACK للخادم
- ينتقل إلى حالة ESTABLISHED

هذه هي خطوة Step 3 في الـ Three-Way Handshake
في هذه الحالة الاتصال أصبح جاهزاً لتبادل البيانات بين الطرفين.

4. FIN_WAIT

عندما يقرر العميل إغلاق الاتصال:

- يرسل FIN إلى الخادم
- ينتقل إلى حالة FIN_WAIT_1
- ينتظر ACK من الخادم

هذه هي خطوة Step 1 في الـ Four-Way Handshake .

5. FIN_WAIT

عندما يستقبل العميل:

- ACK من الخادم على الـ FIN
- ينتقل إلى حالة FIN_WAIT_2
- ينتظر الخادم ليُرسل FIN الخاص به

هذه هي خطوة Step 2 في الـ Four-Way Handshake .

6. TIME_WAIT

عندما يستقبل العميل:

- FIN من الخادم
- يرسل العميل ACK
- ينتقل إلى حالة TIME_WAIT

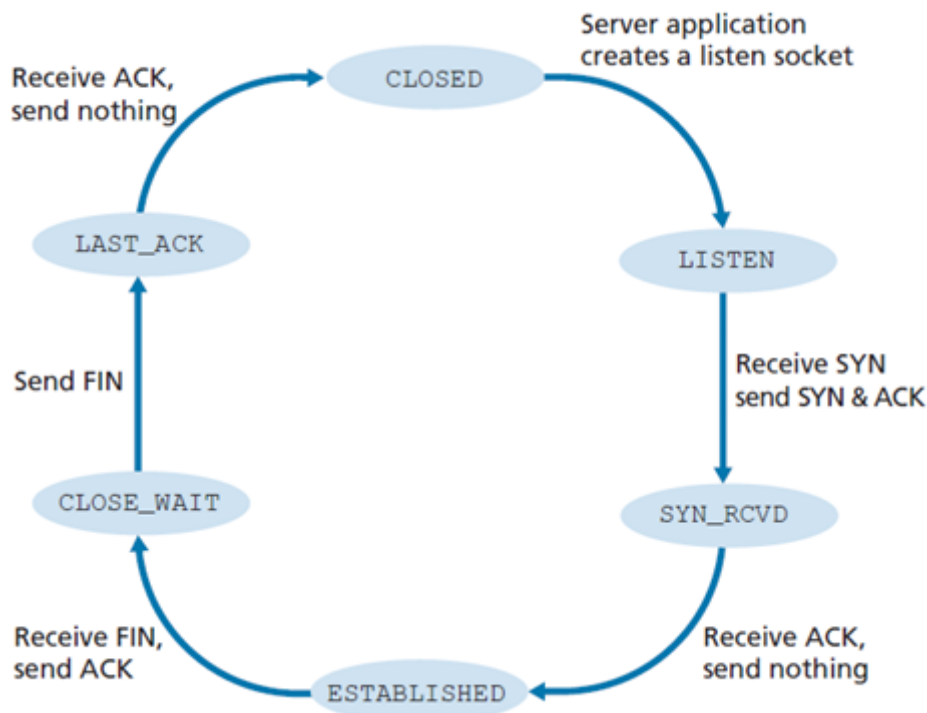
في هذه الحالة:

- ينتظر العميل فترة زمنية (عادة 30 ثانية أو $2 \times \text{MSL}$)
- الهدف هو التأكد من أن ACK الأخير وصل للخادم
- ومنع ظهور Segments متأخرة من اتصال قديم

7. CLOSED (مرة أخرى)

بعد انتهاء فترة TIME_WAIT يتم تحرير جميع الموارد ثم يعود العميل إلى حالة CLOSED

A Typical Sequence of TCP States Visited by a Server-Side TCP



عندما ينتظر الخادم اتصالاً من عميل، ثم يستقبل الاتصال ويُنهيهِ، فإنه يمرّ بمجموعة من الحالات وفقاً لـ TCP FSM هذه الحالات تضمن أن الاتصال يتم بشكل موثوق ومنظم.

1. CLOSED

هذه هي الحالة الابتدائية للخادم حيث لا يوجد اتصال، لا توجد منافذ مفتوحة، لا توجد موارد مخصصة الخادم ينتظر أن يقوم التطبيق باستدعاء `listen()`.

2. LISTEN

عندما يستدعي التطبيق في الخادم الدالة `listen()`

- ينتقل الخادم إلى حالة LISTEN
 - يصبح جاهزاً لاستقبال طلبات الاتصال من العملاء
 - ينتظر وصول SYN من أي عميل
- هذه هي نقطة البداية لإنشاء الاتصال.

3. SYN_RCVD

عندما يستقبل الخادم SYN من العميل يقوم الخادم بـ:

- إرسال SYN + ACK للعميل
- الانتقال إلى حالة SYN_RCVD
- انتظار ACK من العميل

هذه هي خطوة Step 2 في الـ Three-Way Handshake .

4. ESTABLISHED

عندما يستقبل الخادم ACK من العميل على الـ SYNACK ، ينتقل الخادم إلى حالة ESTABLISHED في هذه الحالة الاتصال أصبح جاهز لتبادل البيانات بين الطرفين.

الخادم الآن:

- يرسل البيانات للعميل
- يستقبل البيانات من العميل
- يعمل بشكل كامل كاتصال TCP نشط

5. CLOSE_WAIT

عندما يستقبل الخادم FIN من العميل هذا يعني أن العميل يريد إنهاء الاتصال فالخادم يقوم بإرسال ACK للعميل والانتقال إلى حالة CLOSE_WAIT

في هذه الحالة:

- الخادم قد يكون لديه بيانات متبقية لإرسالها

■ ينتظر التطبيق في الخادم أن يستدعي close()

6. LAST_ACK

عندما يستدعي التطبيق في الخادم close()

■ يرسل الخادم FIN إلى العميل

■ ينتقل إلى حالة LAST_ACK

■ ينتظر ACK من العميل

هذه هي خطوة FIN من الخادم في الـ Four-Way Handshake

7. CLOSED (مرة أخرى)

عندما يستقبل الخادم ACK من العميل على الـ FIN ينتقل الخادم إلى حالة CLOSED وفي هذه اللحظة يتم تحرير جميع الموارد وينتهي الاتصال بالكامل.

ماذا يحدث عندما يستقبل المضيف TCP segment لا ينتمي لأي اتصال نشط لديه—أي لا يطابق أي socket موجود؟

عندما يستلم جهاز (Host) حزمة TCP ، فإنه يقوم بفحص تفاصيلها (رقم المنفذ Destination Port ، وعنوان الـ IP المصدر) ليرى ما إذا كانت تطابق أي اتصال قائم حالياً أو أي خدمة تنتظر اتصالات جديدة.

مثال:

إذا وصل SYN إلى المنفذ 80، لكن الجهاز لا يشغل Web Server على هذا المنفذ، فلا يوجد Socket مرتبط بهذا المنفذ .

في هذه الحالة، لا يمكن للجهاز تجاهل الحزمة تماماً، بل يجب عليه الرد بطريقة تخبر المرسل بأن المحاولة فاشلة.

فإنه يرسل Segment خاص إلى المصدر يسمى Reset Segment ويتم ضبط بت الـ (Reset RST flag bit) في رأس الحزمة (Header) لتكون قيمته 1.

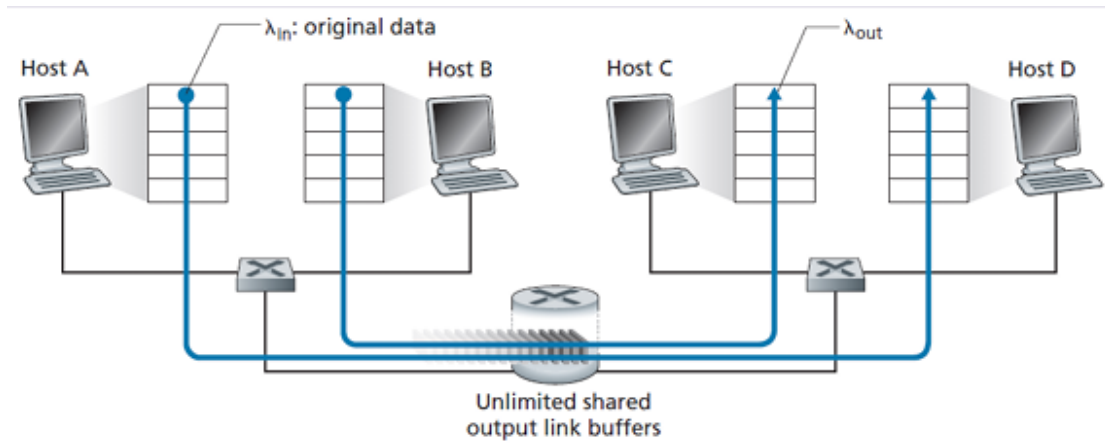
وهذا الـ RST يعني: "ليس لدي مقبس (Socket) لهذا الطلب، يرجى التوقف عن المحاولة وعدم إعادة إرسال هذه الحزمة مرة أخرى".

لماذا نحتاج RST ؟

لمنع إعادة إرسال بيانات غير مرغوب وإنشاء اتصالات غير صحيحة أو استهلاك موارد الجهاز بلا داع أو حتى حدوث أخطاء في التطبيقات.

مبادئ التحكم في الازدحام (Principles of Congestion Control)

السيناريو الأول: مرسلان، وجهاز توجيه (Router) بذاكرة تخزين مؤقت (Buffers) غير محدودة

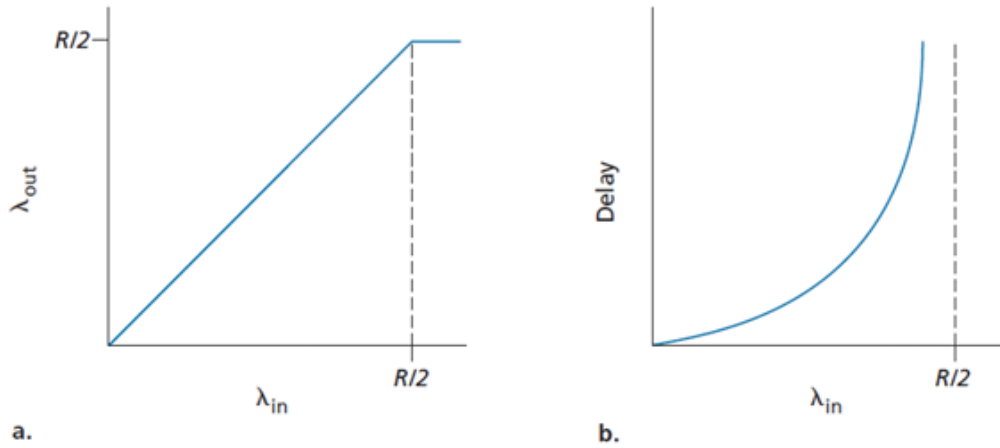


- يرسل المضيفان (A) و (B) بيانات إلى الشبكة بمعدل متوسط قدره λ_{in} بايت/ثانية.
- تمر الحزم القادمة من المضيفين (A) و (B) عبر جهاز توجيه، ومن خلال رابط إرسال مشترك (Shared Outgoing Link) سعته R .
- نفترض أن جهاز التوجيه يمتلك مساحة تخزين مؤقت (Buffer) غير محدودة.

بما أن الذاكرة المؤقتة (Buffer) غير محدودة، فمن المستحيل أن تضع أي حزمة بيانات (No Packet Loss) ولكن إذا تجاوز معدل الإرسال المشترك من الجهازين سعة القناة (أي إذا كان $\lambda_{in} + \lambda_{in} > R$) فإن:

- الراوتر يستقبل حزمًا أكثر مما يستطيع إرسال
- الحزم تتراكم في الـ queue
- التأخير (delay) يبدأ بالارتفاع تدريجيًا
- ثم فجأة ينفجر ويصبح ضخماً جداً

هذه الظاهرة تسمى: Queueing Delay Explosion .



الصورة الأولى توضّح العلاقة بين:

- λ_{in} معدل إرسال المضيفين
- λ_{out} معدل الخروج من الراوتر (Throughput)

1. عندما يكون λ_{in} صغيراً:

- الراوتر يستطيع إرسال كل الحزم
- throughput يساوي λ_{in}
- الخط يكون خطياً

2. عندما يصل λ_{in} إلى $R/2$:

- يبدأ النظام بالوصول إلى نقطة حرجية
- throughput يصل إلى الحد الأقصى الممكن في هذا السيناريو
- بعد هذه النقطة، throughput لا يزيد رغم زيادة λ_{in}

3. عندما يتجاوز λ_{in} قيمة $R/2$:

- throughput يصبح ثابتاً
- أي زيادة في الإرسال لا تزيد الإنتاجية
- كل الزيادة تتحول إلى ازدحام + تأخير

الصورة الثانية توضّح كيف يتغيّر التأخير:

1. عندما يكون λ_{in} صغيراً

- التأخير قليل جداً
- الراوتر يعالج الحزم بسرعة

2. عندما يقترب λ_{in} من $R/2$

- التأخير يبدأ بالارتفاع
- ثم يرتفع بشكل حاد جداً

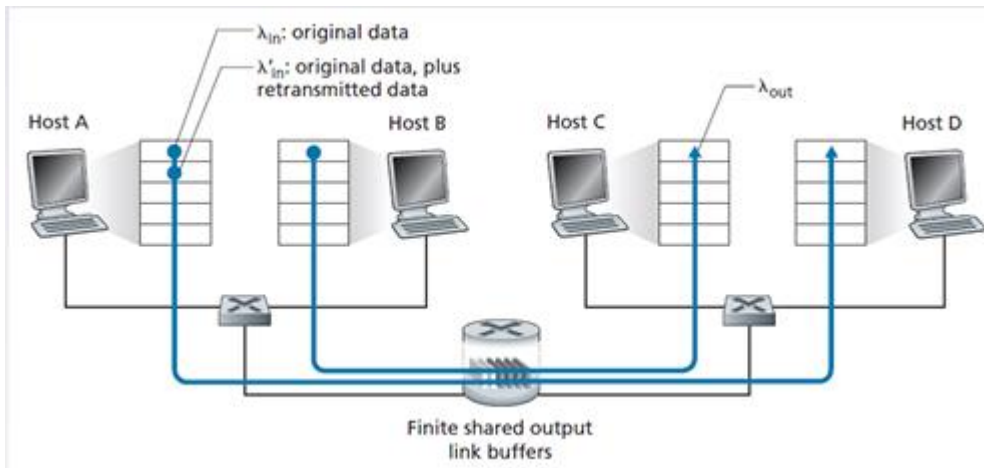
3. عندما يتجاوز λ_{in} $R/2$

- التأخير يصبح ضخماً
- قد يصل إلى ثوانٍ أو دقائق
- رغم أن throughput لم يتحسن إطلاقاً

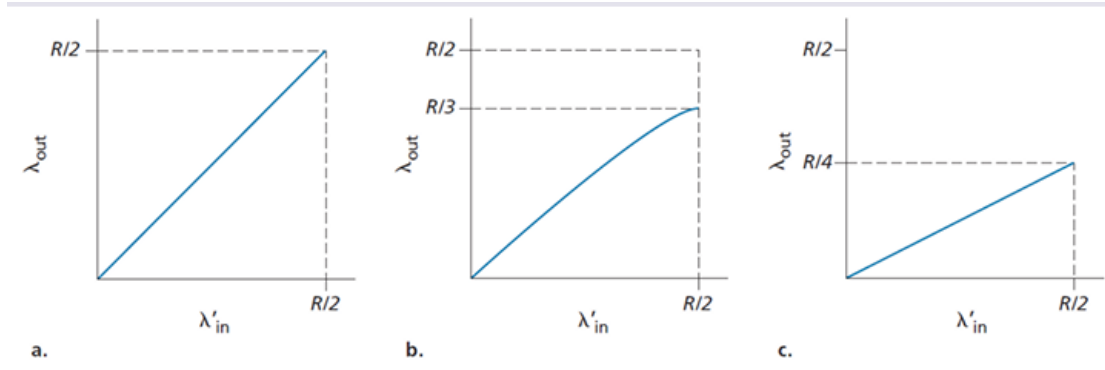
الخلاصة:

مع اقتراب معدل وصول الحزم من سعة الرابط، throughput يصل إلى حدّه الأقصى ولا يزيد، بينما يرتفع التأخير بشكل كبير بسبب تراكم الحزم في queue هذا يوضح أن الازدحام لا يعني فقط فقدان الحزم، بل أيضاً تأخيراً هائلاً

السيناريو الثاني: مرسلان، وجهاز توجيه بذاكرة تخزين مؤقت محدودة



في هذا السيناريو الراوتر يمتلك Buffer محدود والاتصال موثوق (TCP) أي حزمة تُفقد سيتم إعادة إرسالها والحمل الفعلي على الشبكة يصبح $\lambda'_{in} = \text{original data} + \text{retransmissions}$ ويسمى Offered Load .



λ_{in} : يمثل البيانات الأصلية فقط (Original data).

λ'_{in} : يمثل المعدل الإجمالي الذي ترسل به طبقة النقل الحزم إلى الشبكة، وهو يشمل البيانات الأصلية مضافاً إليها البيانات المُعاد إرسالها .

هذه الرسوم الثلاثة توضح كيف يتغير λ_{out} throughput عندما يزيد الحمل الفعلي λ'_{in} الذي يقدمه المرسل للشبكة.

(a) لا يوجد فقدان حزم

في هذا السيناريو لا يحدث فقدان للحزم ولا ازدحام ولا حتى إعادة إرسال لذلك:

$$\lambda_{in'} = \lambda_{in}$$

والـ throughput :

$$\lambda_{out} = \lambda_{in}$$

الخط يكون خطاً مستقيماً من الأصل حتى $R/2$.

ماذا يعني هذا؟

كل ما يرسله المرسل يصل إلى المستقبل، ولا يوجد أي تكلفة إضافية بسبب الازدحام.

(b) إعادة إرسال عند التأكد من فقدان الحزمة

في هذا السيناريو الراوتر يسقط الحزم فقط عندما يمتلئ الـ buffer والمرسل يعيد الإرسال فقط عندما يتأكد 100% أن الحزمة ضاعت ولا توجد إعادة إرسال غير ضرورية لكن بعض الحزم تضيع بالفعل لذلك:

$$\lambda_{out} < \lambda_{in'}$$

والمنحنى يصبح منحنى غير خطي وينتهي عند:

$$\lambda_{out} = R/3$$

ماذا يعني هذا؟

الازدحام يقلل throughput لأن بعض الحزم تضيع ويجب إعادة إرسالها، لكن لا يوجد إعادة إرسال غير ضرورية.

(c) إعادة إرسال غير ضرورية بسبب التأخير الكبير

هذا هو السيناريو الأخطر فالتأخير في الراوتر يصبح كبيراً والمرسل يعتقد أن الحزمة ضاعت لكن الحقيقة أنها موجودة في الـ queue ولم تفقد فيعيد إرسالها قبل أن تصل وفي النهاية كل حزمة قد ترسل مرتين في المتوسط (الأصلية + نسخة غير ضرورية)

لذلك:

$$\lambda_{in}' = 2\lambda_{in}$$

والـ throughput ينخفض أكثر:

$$\lambda_{out} = R/4$$

ماذا يعني هذا؟

الازدحام لا يسبب فقط فقدان حزم، بل يسبب إعادة إرسال غير ضرورية، وهذا يستهلك عرض النطاق في الراوتر ويزيد الازدحام أكثر وأكثر.

تكاليف الشبكة المزدحمة (Congested Network Costs)

1. Retransmissions بسبب فقدان الحزم (Packet Loss)

عندما يمتلئ الـ buffer الراوتر يسقط الحزم و TCP يعيد إرسالها، هذا يزيد الحمل على الشبكة ويقلل throughput

2. Retransmissions غير ضرورية بسبب التأخير الكبير

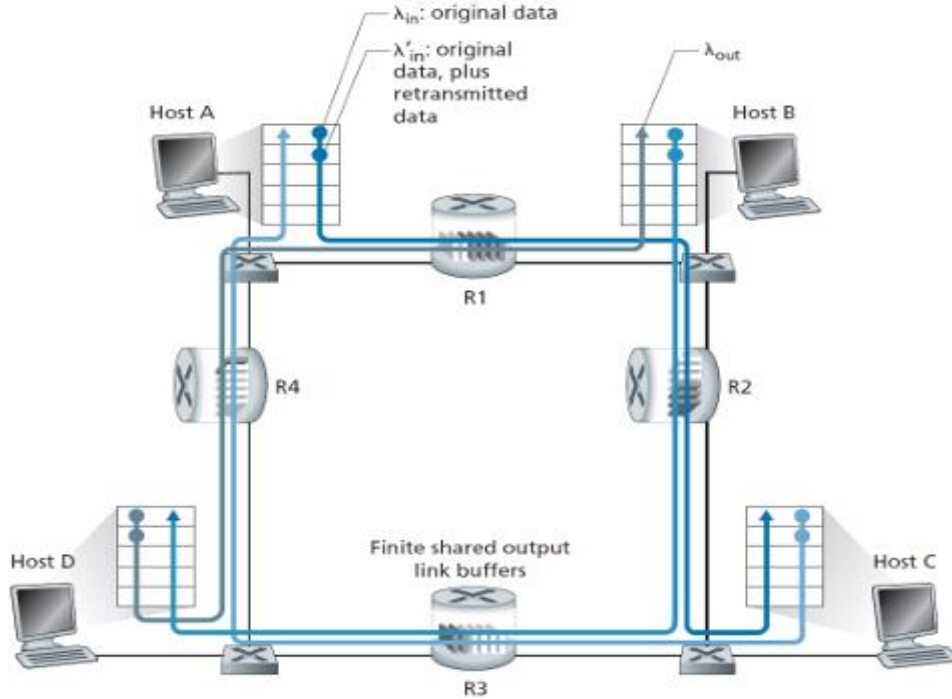
كما ذكرنا سابقاً فعندما يصبح التأخير كبيراً المرسل يعتقد أن الحزمة ضاعت فيعيد إرسالها بينما الحزمة الأصلية ما زالت في الـ queue ، فالراوتر يضطر لإرسال نسختين من نفس الحزمة وهذا يستهلك عرض النطاق بلا فائدة ويزيد الازدحام أكثر، وفي هذه الحالة قد تؤدي إلى Congestion Collapse:

حيث:

- المرسل يرسل الكثير
- الراوتر يعيد توجيه الكثير
- لكن المستقبل يستقبل القليل جداً
- throughput ينهار رغم أن الشبكة مشغولة بالكامل



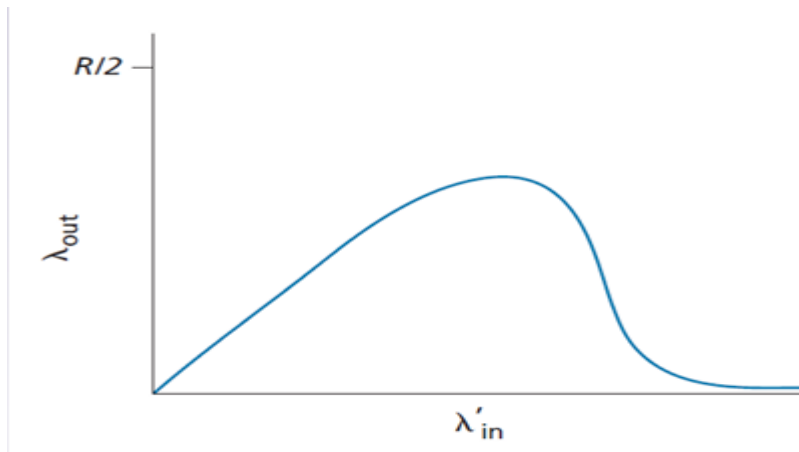
السيناريو الثالث: أربعة مرسلين، أجهزة توجيه بذاكرة محدودة، ومسارات متعددة القفزات (Multihop Paths)



لدينا شبكة فيها أربعة مضيفين A ، B ، C ، D وكل مضيف يرسل بيانات عبر مسارين من خطوتين (Two-hop paths) على سبيل المثال:

- حركة مرور الجهاز A تمر عبر الراوتر R1 ثم R2 لتصل للمستقبل.
 - حركة مرور الجهاز B تمر عبر R2 ثم R3 ، وهكذا في حلقة دائرية متداخلة.
- ولكن المسارات متداخلة (overlapping) وكل راوتر لديه Buffer محدود وكل رابط لديه سعة: R bytes/sec

كل مضيف يرسل بمعدل λ_{in} وكل مضيف يستخدم Timeout + Retransmission لضمان الموثوقية (TCP)



1. عندما يكون λ_{in} صغيراً جداً

- لا يوجد ازدحام
- لا يوجد فقدان حزم
- لا يوجد إعادة إرسال

throughput يساوي offered load : $\lambda_{out} \approx \lambda_{in}$ لأن الـ buffers لا تمتلئ ، والراوترات تعمل بسلاسة.

2. عندما يكون λ_{in} صغيراً إلى متوسط

- بعض الازدحام يبدأ بالظهور
- لكن معظم الحزم تصل
- throughput يستمر بالارتفاع مع زيادة λ_{in}
- الشبكة تعمل بشكل جيد

3. عندما يصبح λ_{in} كبيراً يبدأ throughput بالانخفاض

هنا تحدث المشكلة Throughput Collapse لكن لماذا ينخفض throughput رغم أن المرسل يرسل أكثر؟ هذا لأن الشبكة تبدأ بعمل عمل غير مفيد (Wasted Work)

ما هو Wasted Work ؟

فكر في هذا المثال:

Host A يرسل حزمة والراوتر الأول يوجّه الحزمة إلى الراوتر الثاني لكن الراوتر الثاني يسقط الحزمة بسبب امتلاء الـ buffer، أي أن كل العمل الذي قام به الراوتر الأول لإرسال الحزمة إلى الراوتر الثاني كان عملاً ضائعاً.

لماذا يؤدي هذا إلى انخفاض throughput ؟

لأن الراوترات تعمل بجهد كبير لكنها تعمل على حزم لن تصل أبداً والمرسل يعيد إرسال الحزم فيزداد الحمل أكثر والراوترات تعيد توجيه نسخ متعددة من نفس الحزمة وفي النهاية λ_{out} ينخفض رغم أن λ_{in} يرتفع وهذا ما يظهر في المنحنى: يرتفع throughput أولاً حتى يصل إلى ذروة ثم ينهار مع زيادة offered load

■ تكلفة الازدحام في هذا السيناريو

1. Retransmissions ← Dropped Packets

عندما تسقط الحزم: TCP يعيد إرسالها فالحمل يزيد والازدحام يزيد وال throughput ينخفض

2. Wasted Work في الراوتر upstream

عندما تُسقط الحزمة في hop متأخر: كل العمل في hops السابقة يصبح بلا فائدة والشبكة تستهلك مواردها في نقل حزم لن تصل وهذا يسبب انهيار الأداء

الخلاصة:

عندما يزداد offered load ، يبدأ throughput بالانخفاض لأن الشبكة تقوم بعمل غير مفيد: الحزم التي تُسقط في hops متأخرة تجعل العمل في hops السابقة ضائعاً. هذا يؤدي إلى انخفاض throughput رغم زيادة الحمل.

انتهت المحاضرة

